

ZYNQ 开发平台 HLS 教程

版本号：V1.03

2024-04-03 09:46:56

版权声明

Copyright ©2012-2018 芯驿电子科技（上海）有限公司

公司网址:

[Http://www.alinx.com.cn](http://www.alinx.com.cn)

技术论坛:

<http://www.heijin.org>

官方旗舰店:

<http://alinx.jd.com>

邮箱:

avic@alinx.com.cn

电话:

021-67676997

传真:

021-37737073

ALINX 微信公众号:



前言

Vivado HLS 能提高系统设计的抽象层次，为设计人员带来切实的帮助。Vivado HLS 通过下面两种方法提高抽象层次：第一，使用 C/C++ 作为编程语言，充分利用该语言中提供的高级结构。第二，提供更多数据原语，便于设计人员使用基础硬件构建块（位向量、队列等）。与使用 RTL 相比，这两大特性有助于设计人员使用 Vivado HLS 更轻松地解决常见的协议系统设计难题。最终简化系统汇编，简化 FIFO 和存储器访问，实现控制流程的抽象。

Vivado HLS 便于架构研究，用户只需在代码中插入程序指令（如使用 GUI 或批处理模式时的 Tcl 命令），就可以把设计所需特性传递给综合工具。这样用户可以在不修改设计代码本身的情况下研究大量备选架构方案。研究的范围可以是模块流水线化等根本性问题，也可以是 FIFO 队列深度等较常见的问题。

Vivado HLS 便于仿真。C 和 RTL 仿真是 Vivado HLS 另一个大放异彩的地方。设计一般采用两步流程验证：第一步是 C 语言仿真。这个步骤中 C/C++ 的编译和执行与常见的 C/C++ 程序相同；第二步是 C/RTL 协仿真。在这一步骤中，Vivado HLS 会根据 C/C++ 测试平台自动生成 RTL 测试平台，然后设置并执行 RTL 仿真，检查实现方案吧的正确性。

如能充分发挥这些优势，这将对于用户的系统设计大有裨益。这不仅体现在开发时间和生产力上，还由于 Vivado HLS 代码更加紧凑的特点，体现在代码可维护性和可读性上。此外通过高层次综合，用户仍能有效控制架构及其特性。正确理解和使用 Vivado HLS 程序对实现这一控制起着根本作用。

目录

版权声明	2
前言	3
目录	4
准备工作及注意事项	7
软件环境	7
硬件环境	7
实验工程及目录说明	10
实验快速复现	11
工程重新编译注意事项	错误! 未定义书签。
第一章 初识 HLS	12
1.1 实验 led 控制	12
1.1.1 创建 vivado hls 工程	12
1.1.2 创建 vivado 工程	18
1.1.3 实验总结	32
1.2 工程路径	32
1.3 HLS 简介	33
1.3.1 Vivado HLS 包含库	33
1.3.2 Vivado HLS 接口	34
1.3.3 hls 官方教程	34
第二章 状态指示 led	35
2.1 模块控制 block-level	35
2.2 可配置的模块	35
2.3 工程路径	36
2.4 实验结果	36
第三章 浮点协处理	错误! 未定义书签。
3.1 实验介绍	错误! 未定义书签。
3.2 IP 创建	错误! 未定义书签。
3.2.1 HLS 源代码	错误! 未定义书签。
3.2.2 接口介绍	错误! 未定义书签。
3.2.3 运算	错误! 未定义书签。
3.2.4 其它说明	错误! 未定义书签。
3.3 TestBench	错误! 未定义书签。
3.3.1 应用程序创建	错误! 未定义书签。
3.3.2 源代码	错误! 未定义书签。

3.3.3 C 仿真	错误! 未定义书签。
3.3.4 RTL 仿真	错误! 未定义书签。
3.4 工程路径	错误! 未定义书签。
3.5 运行结果	错误! 未定义书签。
第四章 视频彩条	37
4.1 Vivado HLS 视频开发	37
4.1.1 与 OpenCV 关系	37
4.1.2 VivadoHLS 视频库函数	37
4.2 实验介绍	38
4.3 HLS IP 创建	38
4.3.1 源代码	38
4.3.2 接口介绍	43
4.3.3 hls::Mat 介绍	43
4.3.4 优化	43
4.4 工程路径	43
4.5 实验结果	44
第五章 视频帧缓存读写管理	45
5.1 实验介绍	45
5.2 模块主要代码	45
5.1 工程路径	49
5.2 实验结果	49
第六章 图像缩放叠加	错误! 未定义书签。
6.1 实验介绍	错误! 未定义书签。
6.2 模块主要代码	错误! 未定义书签。
6.3 工程路径	错误! 未定义书签。
6.4 实验结果	错误! 未定义书签。
第七章 字符叠加	错误! 未定义书签。
7.1 实验介绍	错误! 未定义书签。
7.2 模块主要代码	错误! 未定义书签。
7.3 工程路径	错误! 未定义书签。
7.4 实验结果	错误! 未定义书签。
第八章 图像对比度调整	51
8.1 实验介绍	51
8.2 模块主要代码	51
8.3 工程路径	55
8.4 实验结果	55
第九章 自动聚焦	错误! 未定义书签。
9.1 实验介绍	错误! 未定义书签。

9.2 代码	错误! 未定义书签。
9.3 工程路径	错误! 未定义书签。
9.4 实验结果	错误! 未定义书签。
第十章 边缘检测	57
10.1 实验介绍	57
10.2 TestBench 结果	57
10.3 模块主要代码	59
10.4 工程路径	64
10.5 实验结果	64
第十一章 角点检测	66
11.1 模块主要代码	67
11.2 工程路径	70
11.3 实验结果	71
第十二章 快速傅里叶变换 FFT	错误! 未定义书签。
12.1 实验介绍	错误! 未定义书签。
12.2 模块主要代码	错误! 未定义书签。
12.3 工程路径	错误! 未定义书签。
12.4 实验结果	错误! 未定义书签。

准备工作及注意事项

本教程基于学习者已经熟练使用 Vivado 和 Vitis 的情况下编写，不详细讲解 Vivado 工程建立，Vitis 裸机软件编写，中断处理等。HLS 要求学习者熟练使用 C++，具备一定硬件思维，对硬件流水线、组合逻辑、时序逻辑、FIFO、RAM、AXI 总线协议有深刻理解才能学习，免得出现低级错误。

软件环境

本教程基于软件版本 Vitis 2023.1，这里要用到的快捷方式图标有：



vivado 软件

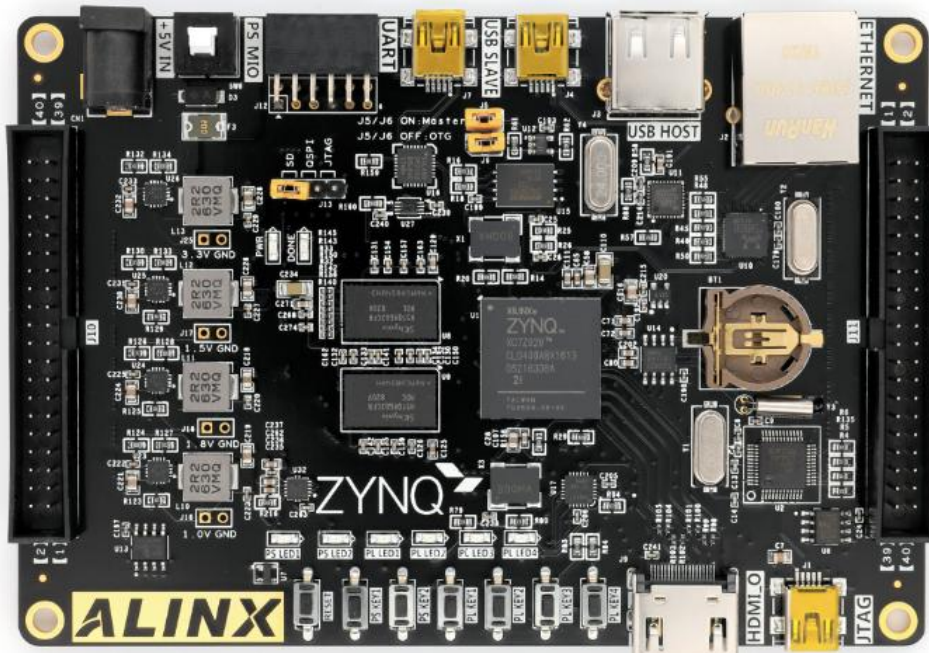


vivado hls 软件

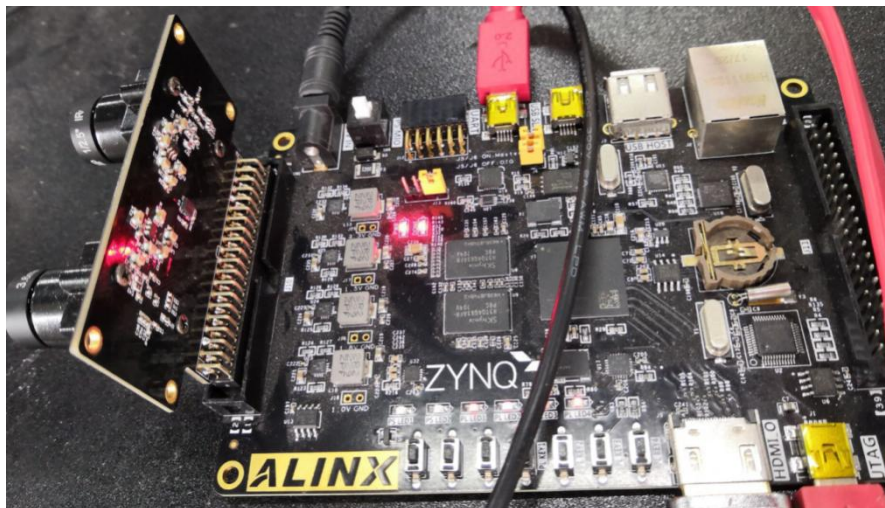
硬件环境

ZYNQ (ALINX 芯驿电子科技) 开发板一块，双目镜头模块一个。当前支持的型号列表如下：

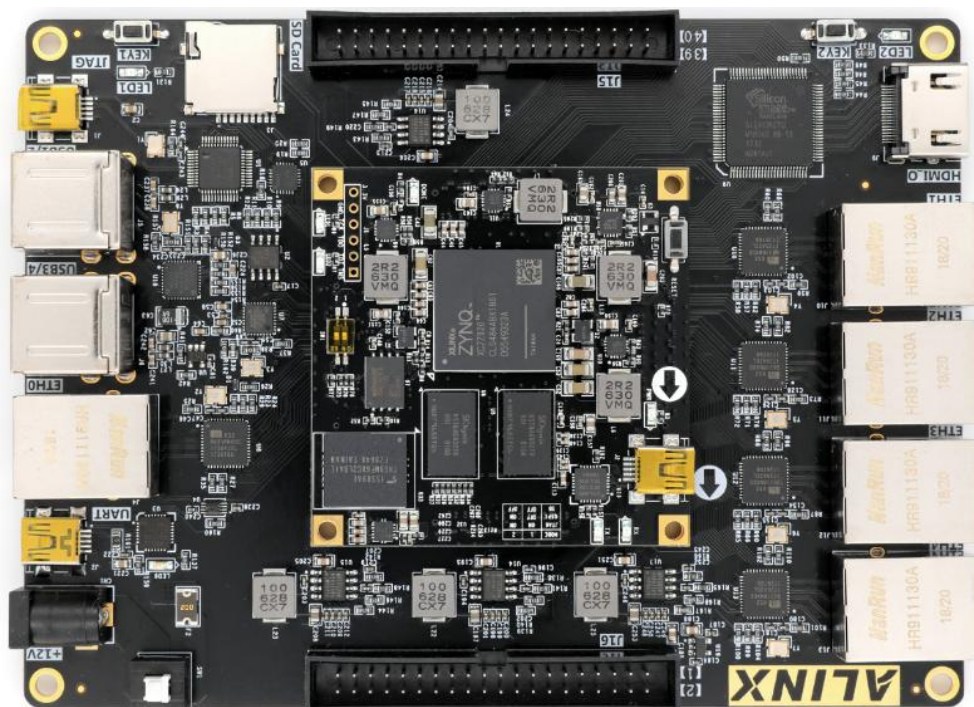
开发板型号	双目模块	ZYNQ 芯片型号	ddr 参数兼容型号
AX7010	AN5642	xc7z020clg400-2	MT41J256M16 RE-125
AX7020	AN5642	xc7z020clg484-2	MT41J256M16 RE-125
AX7015	AN5642	xc7z015clg485-2	MT41J256M16 RE-125
AX7350	FL0214	xc7z035ffg676-2	MT41J256M16 RE-125
AX7Z035	AN5642	xc7z035ffg676-2	MT41J256M16 RE-125
AX7Z100	AN5642	xc7z100ffg900-2	MT41J256M16 RE-125



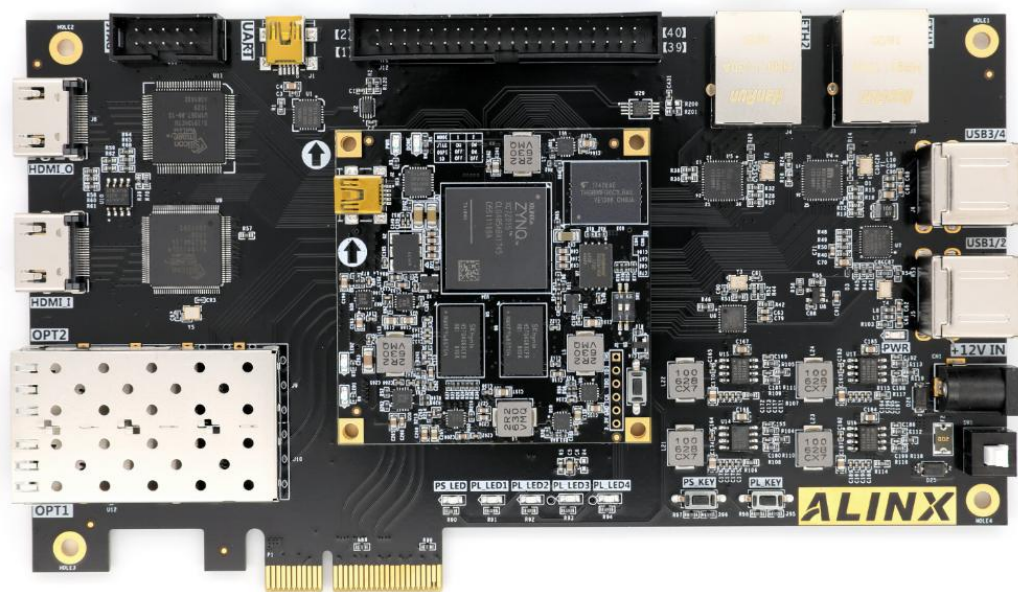
AX7020



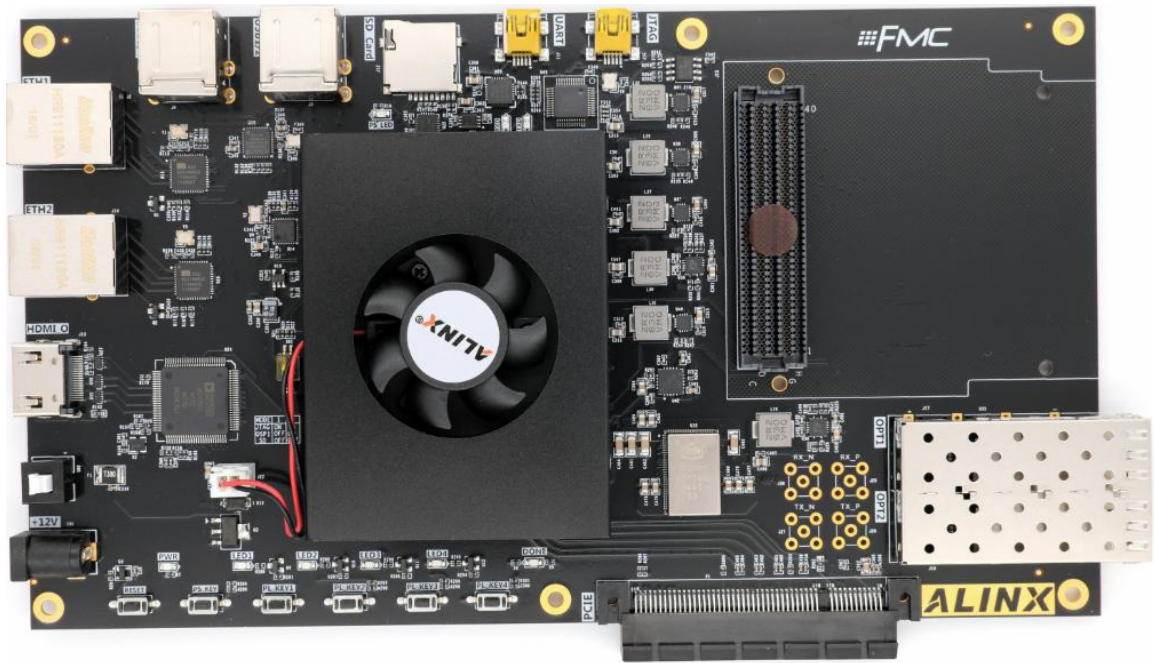
AX7020 正确连接摄像头方式，例程中都是插在 J10



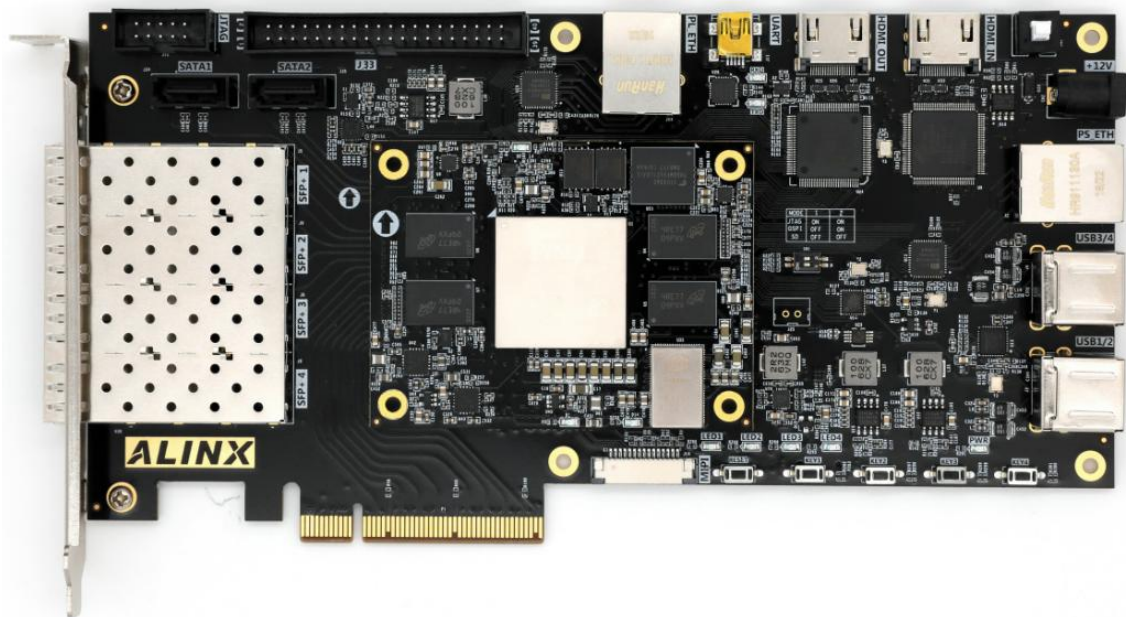
AX7021



AX7015



AX7350



AX7Z035/AX7Z100

另外需要自己准备好 HDMI 显示器一台（支持 1080P60）。

实验工程及目录说明

工程名称	目录路径	说明
HLS 工程	hls	各算法导出 IP，由 vivado 引用
vivado 工程	vivado	
vitis 工程	vitis	软件程序
常用 ip	vivado/xx_project/ip/normal	Verilog 封装 ip

实验快速复现

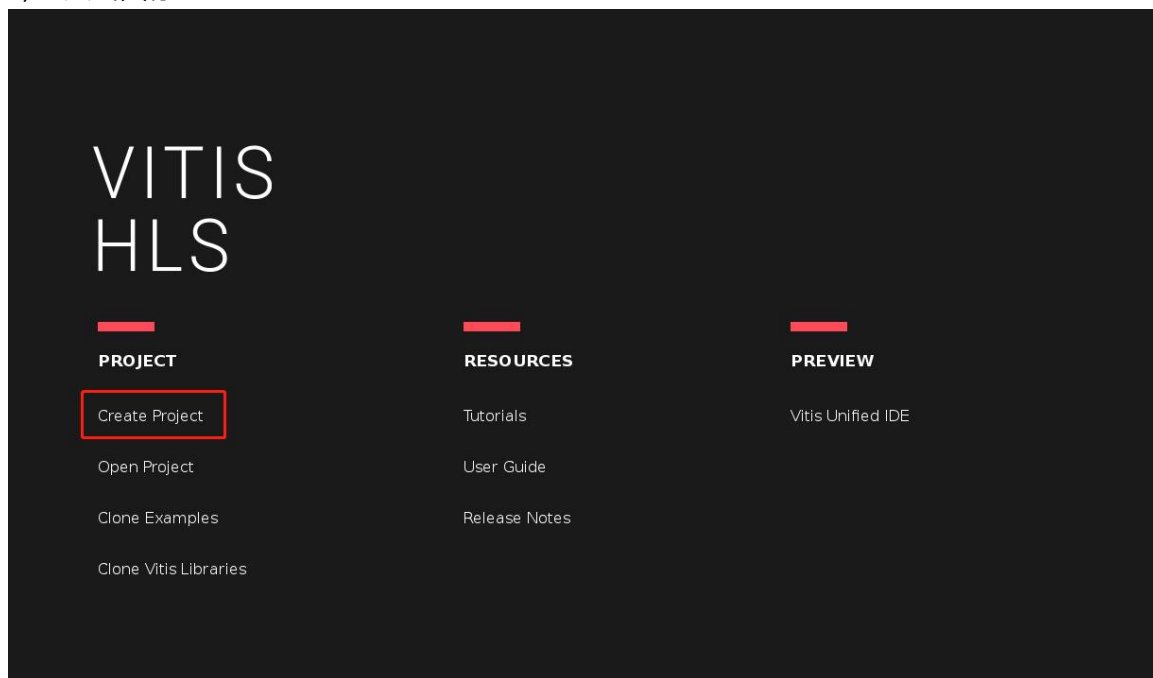
- 1) 连接好 jtag 接口
- 2) 设置启动模式为 QSPI 启动
- 3) 将对应实验 SDK 工程中文件 BOOT.bin 拷贝到 flash_load_comm 目录下。
- 4) 开发板上电

第一章 初识 HLS

1.1 实验 led 控制

1.1.1 创建 vitis hls 工程

- 1) 打开 vitis hls 软件
- 2) 点击图标



- 3) 设置工程名称及选择存放路径，如图

New Vitis HLS Project @alinx-System-Product-Name

Project Configuration

Create Vitis HLS project of selected type

Project name:

Location:

4) 设置顶层函数名，如图

New Vitis HLS Project @alinx-System-Product-Name

Add/Remove Design Files

Add/remove C-based source files (design specification)

Top Function:

Design Files

Name	CFLAGS	CSIMFLAGS
------	--------	-----------

- 5) Next→Next 至如下界面, 在 Clock Period 一栏可填入运行时钟的周期, 软件会根据此时钟进行逻辑优化, 可根据实际需要填写, 默认是 10ns, clock uncertainty 默认为时钟周期的 12.5%。

New Vitis HLS Project @alinx-System-Product-Name

Solution Configuration

Create Vitis HLS solution for selected technology

Solution Name:

Clock

Period: Uncertainty:

Part Selection

Part: *xcvu11p-flga2577-1-e*

- 6) 点击图标

New Vitis HLS Project @alinx-System-Product-Name

Solution Configuration

Create Vitis HLS solution for selected technology

Solution Name:

Clock

Period: Uncertainty:

Part Selection

Part: *xcvu11p-flga2577-1-e* ...

Flow Target

▼ Configure [several options](#) for the selected flow target

< Back Cancel Finish

7) Search 后输入框中输入 ZYNQ 芯片型号，在下拉列表框中选中对应器件，点击 “OK”按钮

Device Selection Dialog @alinx-System-Product-Name

Select:

Filter

Product Category: All Package: All

Family: zynq Speed grade: All

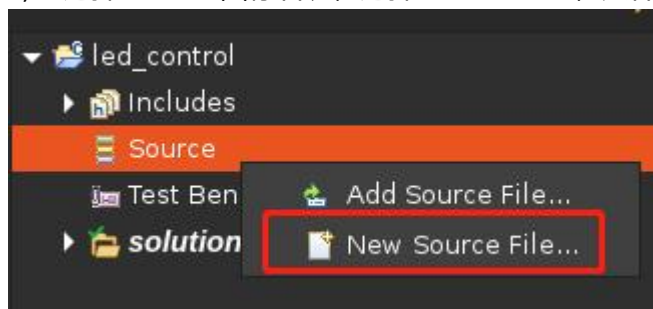
Sub-Family: All Temp grade: All

Search:

Part	Family	Package	Speed	SLICE	LUT
xc7z010iclg400-1L	Zynq-7000	clg400	-1L	4400	17600
xc7z010iclg225-1L	Zynq-7000	clg225	-1L	4400	17600
xc7z010iclg400-3	Zynq-7000	clg400	-3	4400	17600
xc7z010iclg400-2	Zynq-7000	clg400	-2	4400	17600
xc7z010iclg400-1	Zynq-7000	clg400	-1	4400	17600
xc7z010iclg225-3	Zynq-7000	clg225	-3	4400	17600
xc7z010iclg225-2	Zynq-7000	clg225	-2	4400	17600

8) 点击 “Finish”按钮，创建 vivado hls 工程完成

9) 选择“Source”图标右键，选择 “New File...”，文件名设置为“lec_control.cpp”，点击“保存”



10) 在 “lec_control.cpp” 文件中输入如下内容

```
#include <ap_int.h>

void led_control(ap_int<1> &led)
{
    #pragma HLS interface ap_none port=led
    #pragma HLS interface ap_ctrl_none port=return
    led = 0;
    for(int i=0; i<50000000; i++)
    {
        if(i==25000000)
            led = ~led;
    }
}
```



```
}
```

11) “Ctrl+S”保存当前文档内容

12) 点击图标，或者点击菜单栏 Solution→Run C Synthesis→Active Solution



13) “Console”栏中输出“Finished C synthesis. ”，综合完成，软件会自动跳出综合报告。在 Timing 信息中，可以看到时钟设置为 10ns，uncertainty 为 1.25ns，软件将目标时钟周期设置为 $10 - 1.25 = 8.75\text{ns}$ 。Estimated 时钟周期（最差路径）为 2.06ns，满足 8.75ns 时钟需求。Latency 需要多少周期输出结果，latency 比 loop 循环多一个周期，为迭代延迟。

Timing Estimate

Target	Estimated	Uncertainty
100.0MHz	328.84MHz	370.37MHz

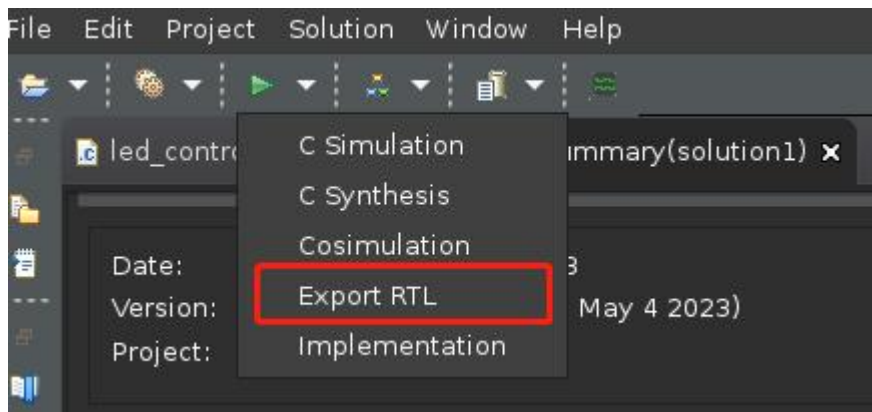
Performance & Resource Estimates

Modules

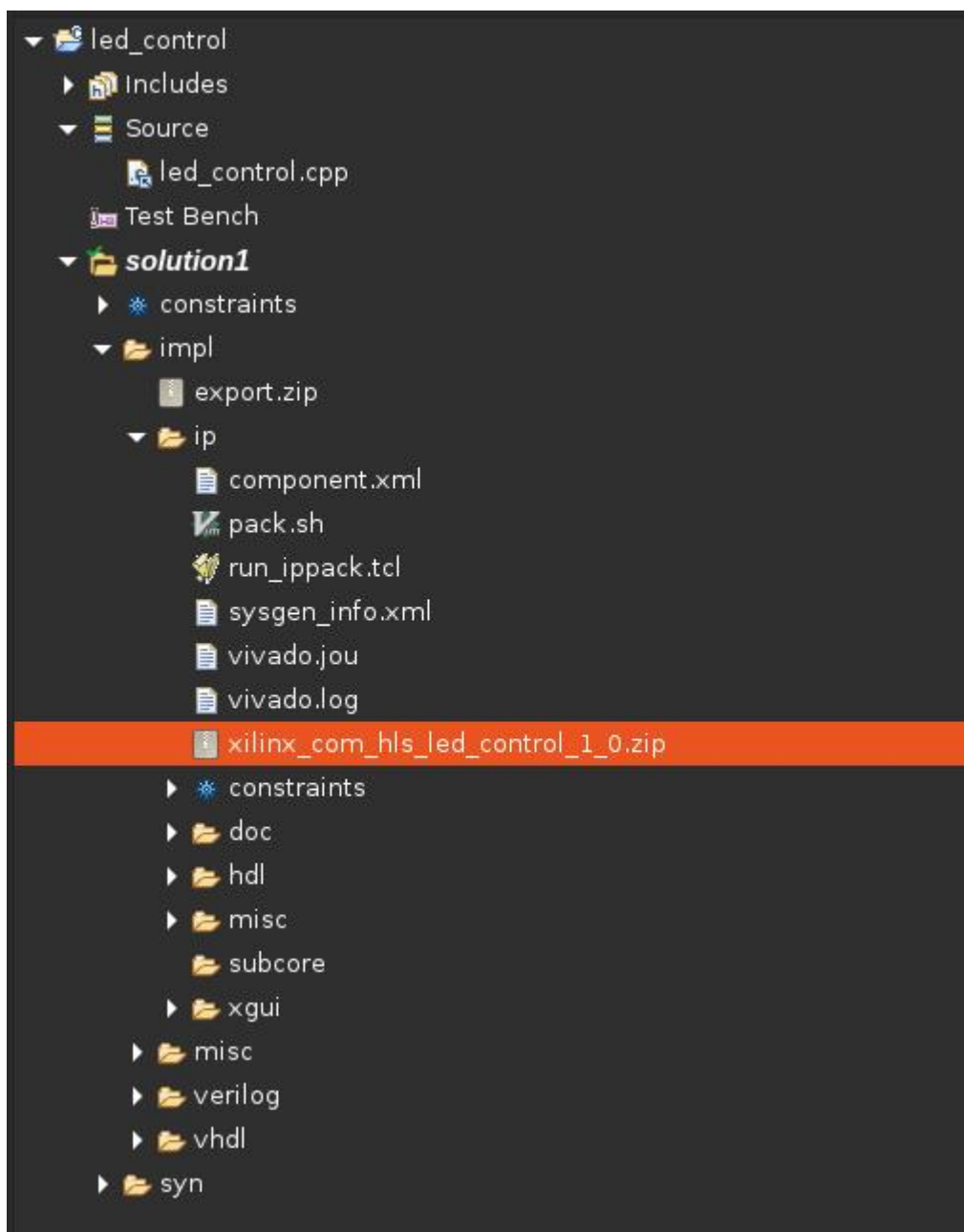
Loops

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
led_control				-	50000002	5.000E8		50000003	-	no	0	0	28	109	0

14) 点击图标，弹出如下对话框，可以在下拉框内选择需要导出的 RTL 类型。



15) “Console”栏中输出“Finished export RTL. ”，导出 IP 完成。在 solution1/impl/ip 文件夹能够找到生成的 IP 压缩包，可在 Vivado 中导入。



1.1.2 创建 vivado 工程

- 1) 打开 vivado 软件
- 2) 点击 “Create Project” 标签，弹出工程创建向导
- 3) Next
- 4) 设置存放路径和工程名
- 5) Next→Next→ Next→ Next, 至如下界面

New Project@alinx-System-Product-Name

Default Part
Choose a default Xilinx part or board for your project.

Parts | Boards

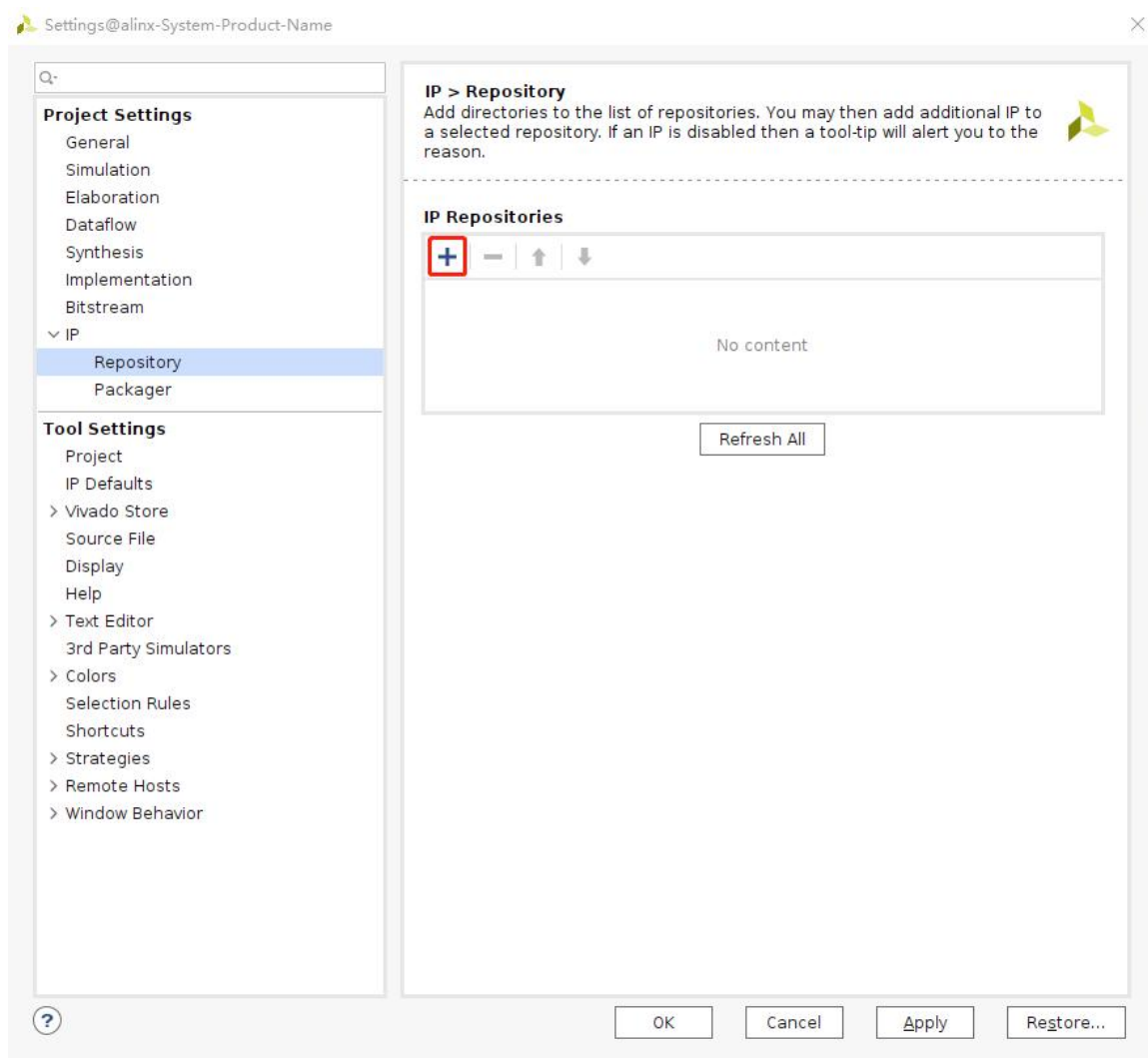
[Reset All Filters](#)

Category: All Package: All Temperature: All
Family: All Speed: All Static power: All

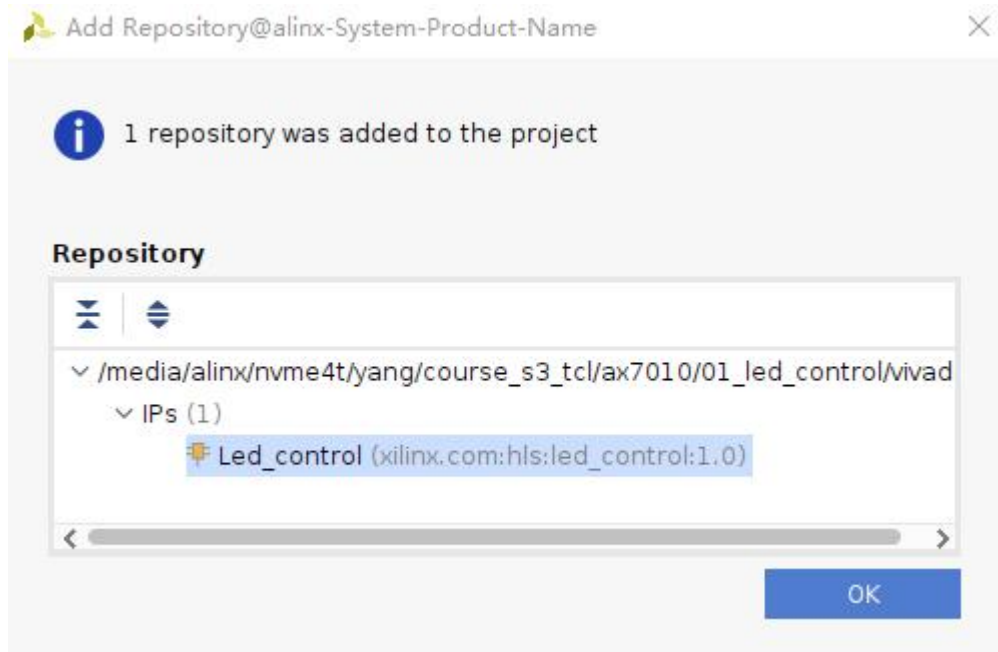
Search: Q-

Part	I/O Pin Count	Available IOBs	LUT Elements	FlipFlops	Block RAMs	Ultra RAMs	DSPs	HNICX	BUFGs	DDR
xc7vx415tffv1157-1	1157	600	257600	515200	880	0	2160		32	
xc7vx415tffv1158-3	1158	350	257600	515200	880	0	2160		32	
xc7vx415tffv1158-2	1158	350	257600	515200	880	0	2160		32	
xc7vx415tffv1158-2L	1158	350	257600	515200	880	0	2160		32	
xc7vx415tffv1158-1	1158	350	257600	515200	880	0	2160		32	
xc7vx415tffv1927-3	1927	600	257600	515200	880	0	2160		32	
xc7vx415tffv1927-2	1927	600	257600	515200	880	0	2160		32	
xc7vx415tffv1927-2L	1927	600	257600	515200	880	0	2160		32	
xc7vx415tffv1927-1	1927	600	257600	515200	880	0	2160		32	
xc7vx485tffg1157-3	1157	600	303600	607200	1030	0	2800		32	
xc7vx485tffg1157-2	1157	600	303600	607200	1030	0	2800		32	
xc7vx485tffg1157-2L	1157	600	303600	607200	1030	0	2800		32	
xc7vx485tffg1157-1	1157	600	303600	607200	1030	0	2800		32	
xc7vx485tffg1158-3	1158	350	303600	607200	1030	0	2800		32	

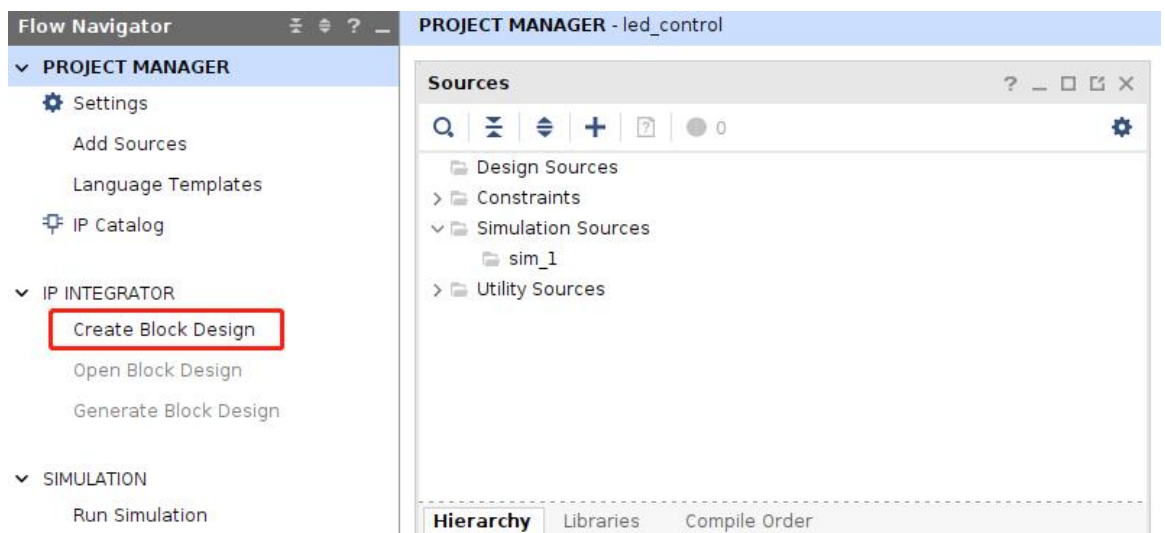
- 6) Search 后输入框中输入 ZYNQ 芯片型号，在下拉列表框中选中对应器件，
- 7) Next→Finish，完成 vivado 工程创建
- 8) 进入到 vivado 软件工程界面，点击如图 “IP Catalog” 标签
- 9) 在下图界面中，右键“Vivado Repository” 标签，选择 “Add Repositories...”，选择 IP 所在路径，点击按钮 “Select”



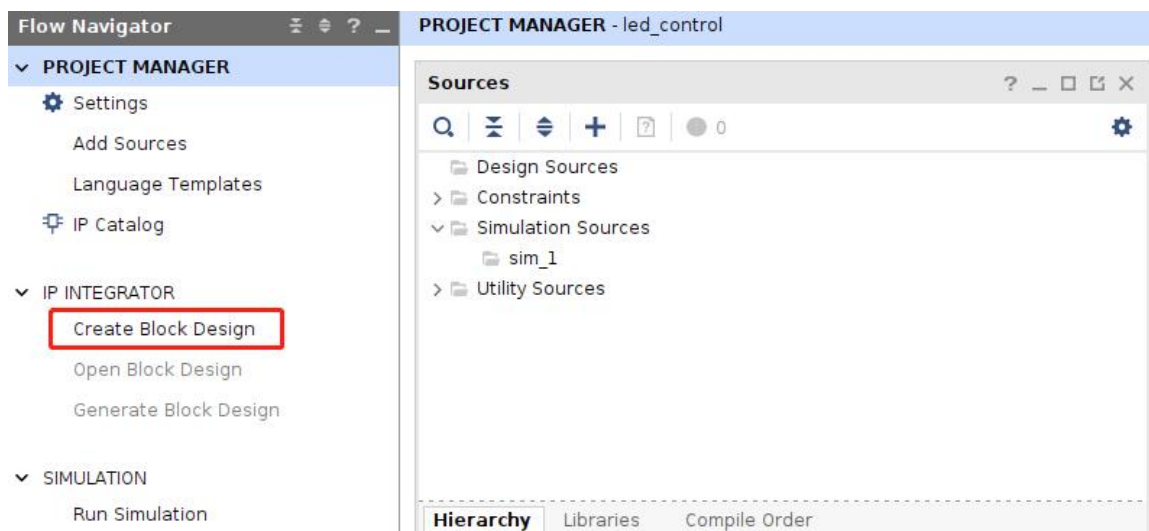
10) 弹出如下界面，点击“OK”，完成 led_control IP 添加



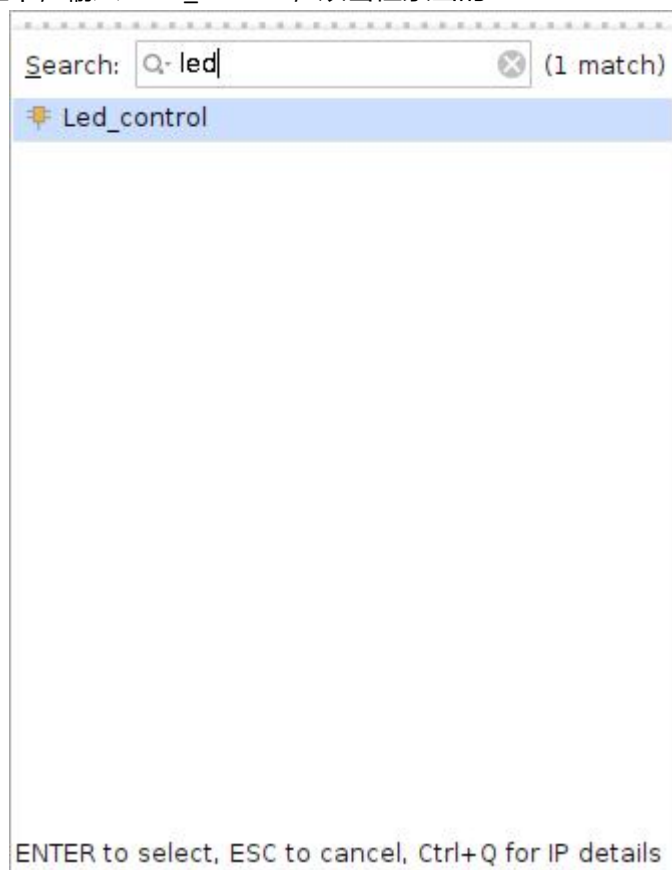
11) 点击图标“Create Block Design”，弹出“Create Block Design”对话框，选择“OK”按钮



12) 在如下界面中，点击如图所示图标，



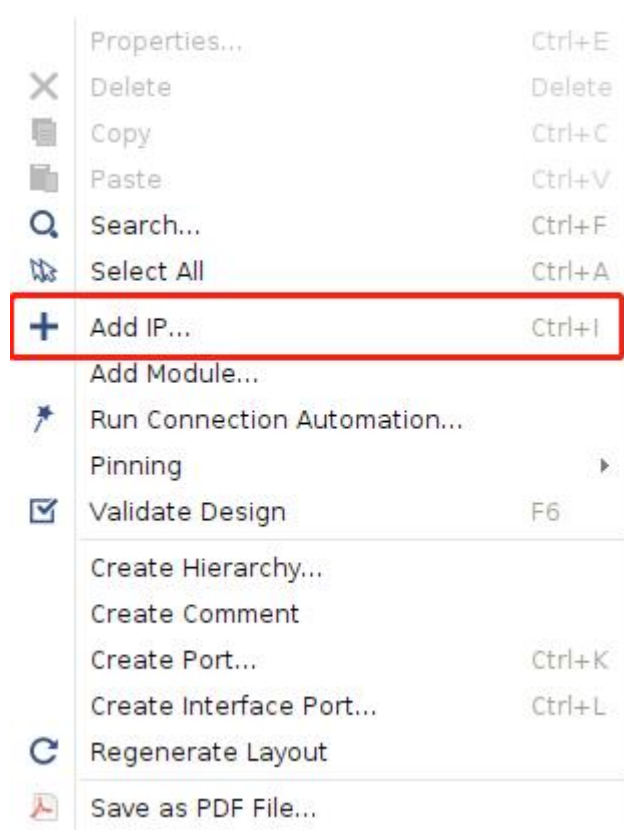
13) 在弹出的对话框中，输入 “led_control”，双击检索出的 IP



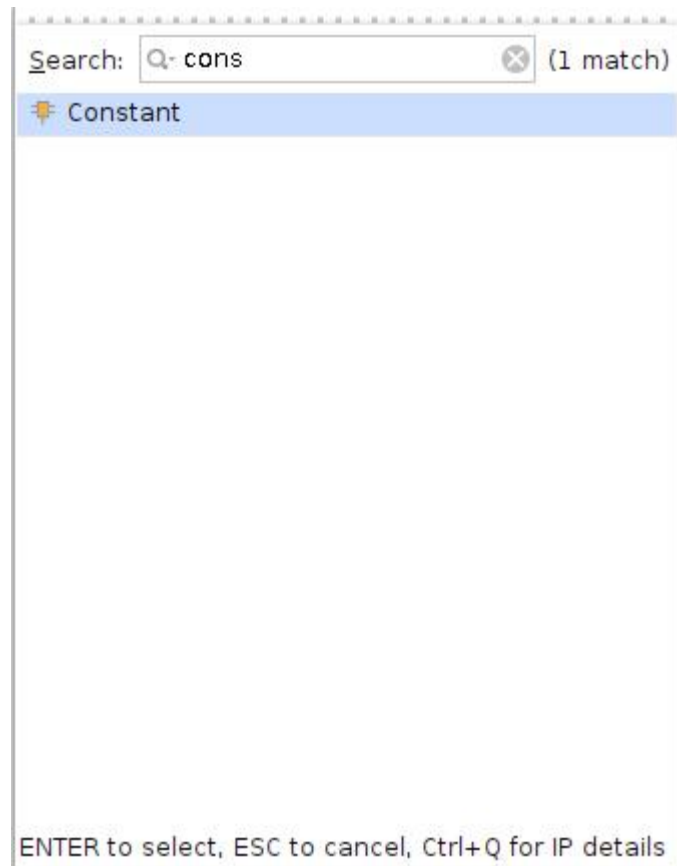
14) 成功在我们设置中，添加了我们设计的 IP，如下图



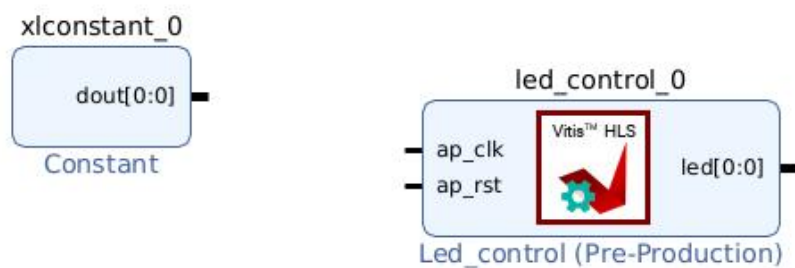
15) 单击右键选择 Add IP



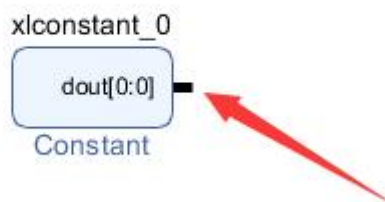
16) 输入“cons”,双击检索出的 IP



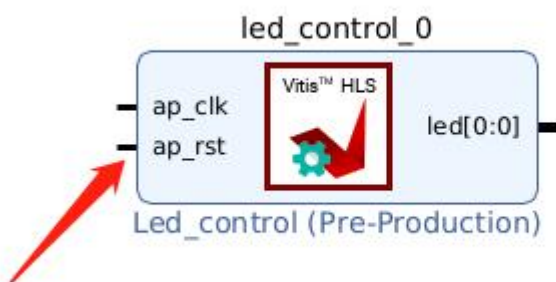
17) 此时我们的设计，如下图



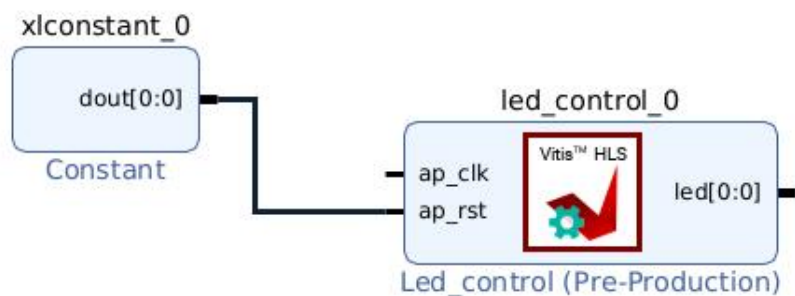
18) 鼠标移至图示管脚上，此时鼠标变成笔状



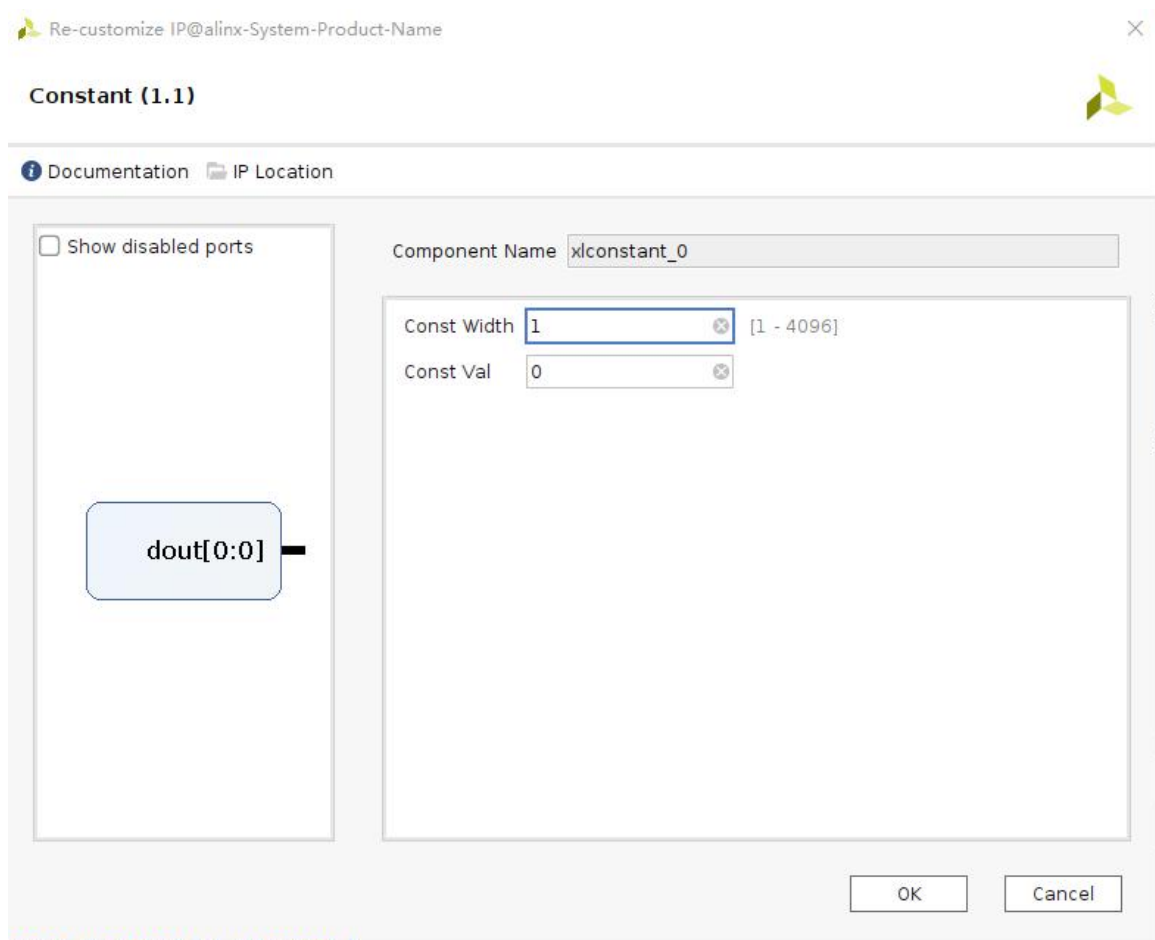
19) 左键鼠标不放，拖动鼠标至下图管脚，会发现两个管脚之间出现连线，此时松开鼠标



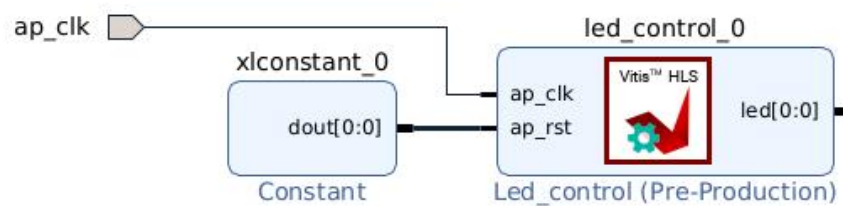
20) 此时我们的设计，如下图



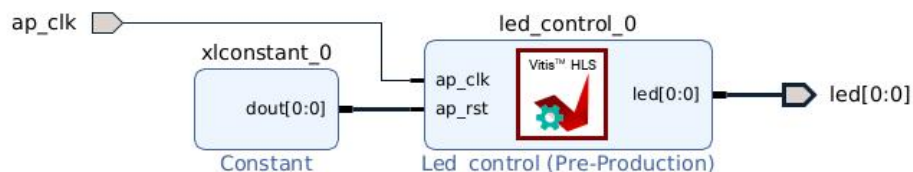
21) 双击图中“xlconstant_0”IP，在弹出的对话框中，修改“Const val”值为 0，点击“OK”。



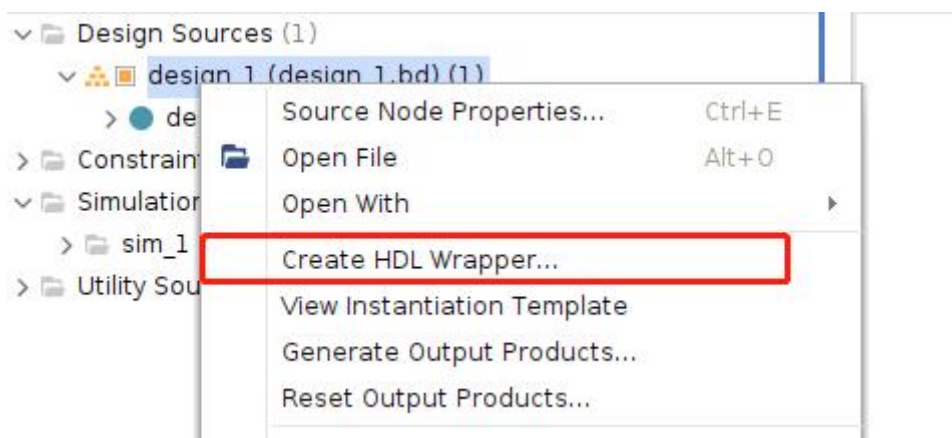
22) 鼠标点击“ap_clk”，选中该管脚，如图所示



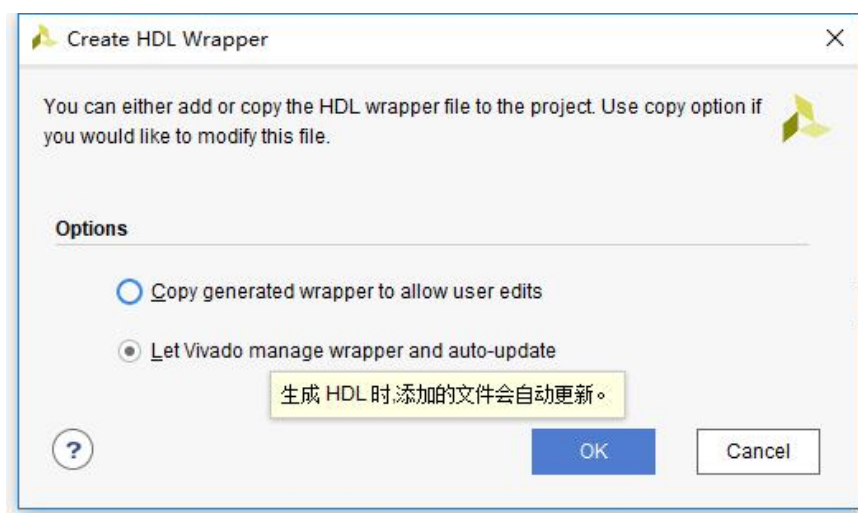
23) 此时，按“Ctrl+T”，相同操作 “led_v[0:0]”管脚，此时设计如下图



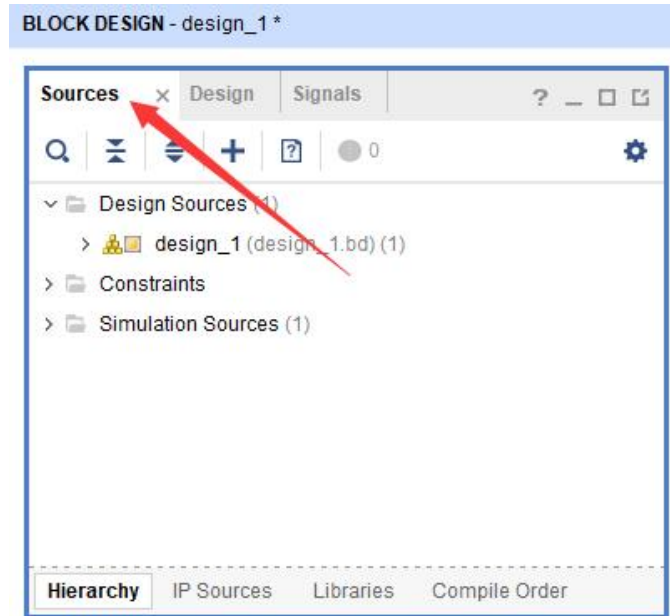
24) 右键 “design_1” ， 选择 “Create HDL Wrapper...”。



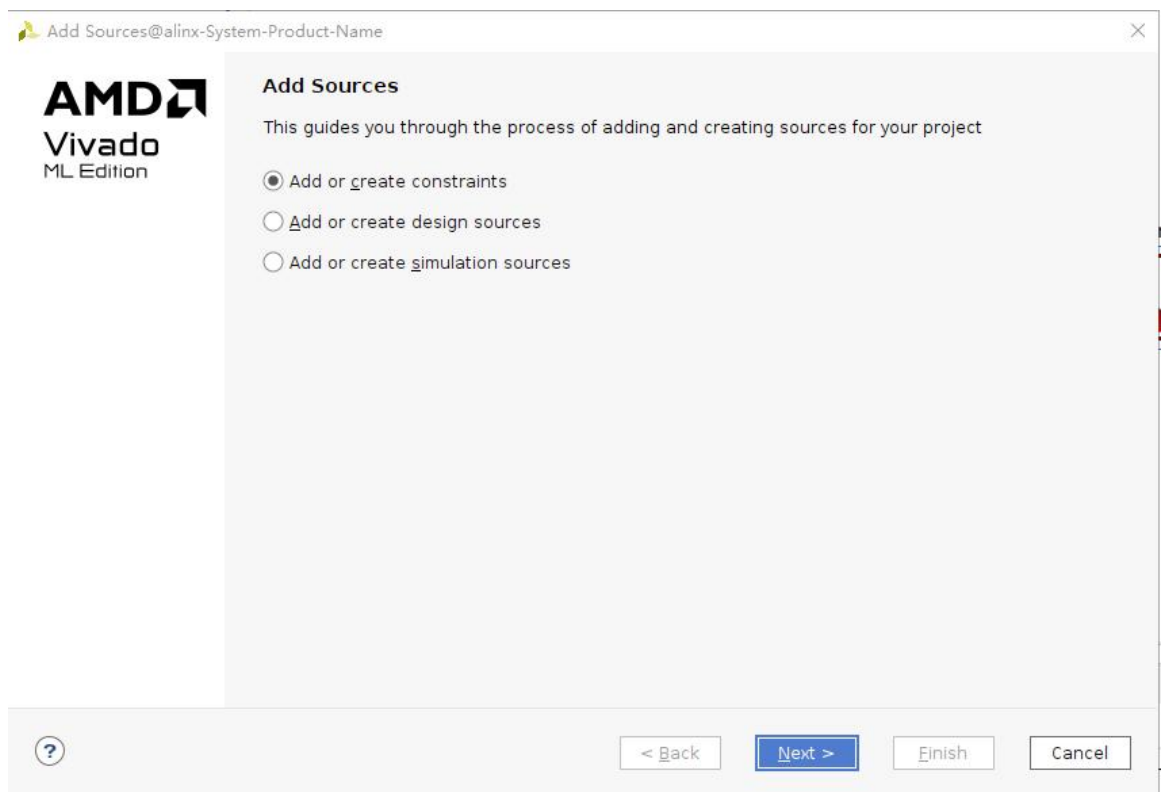
25) 弹出的对话框不作修改，点击 “OK”， ， 生成顶层文件



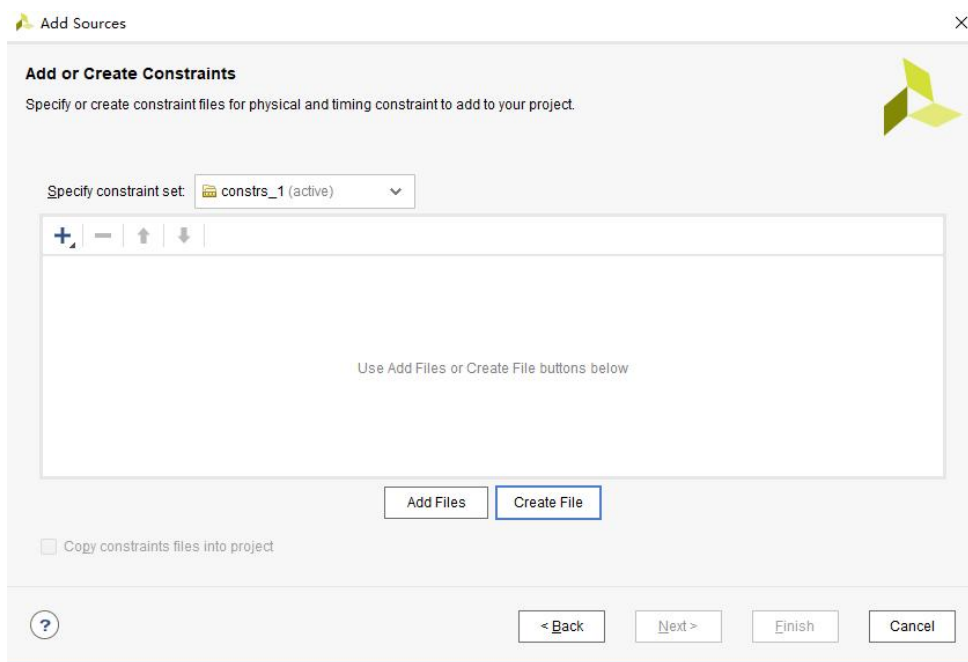
26) 接下来，我们需要为设计中的 “ap_clk_0” 及 “led_v_0[0]” 具体分配物理管脚。若当前没有处在 “Sources” 标签项下，则需要点击 “Sources”，如图



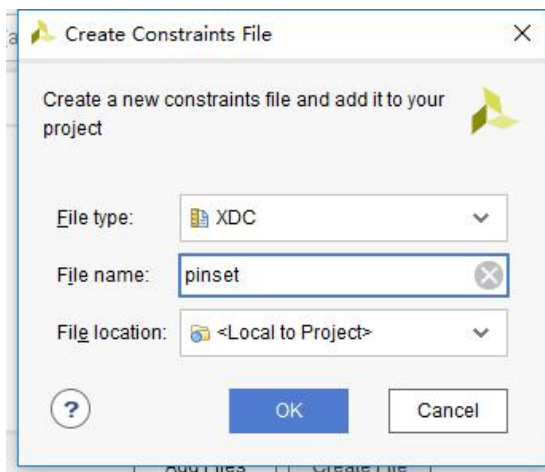
27) 右键图中“Constraints”，在菜单中选择 “Add Sources...”，弹出向导。



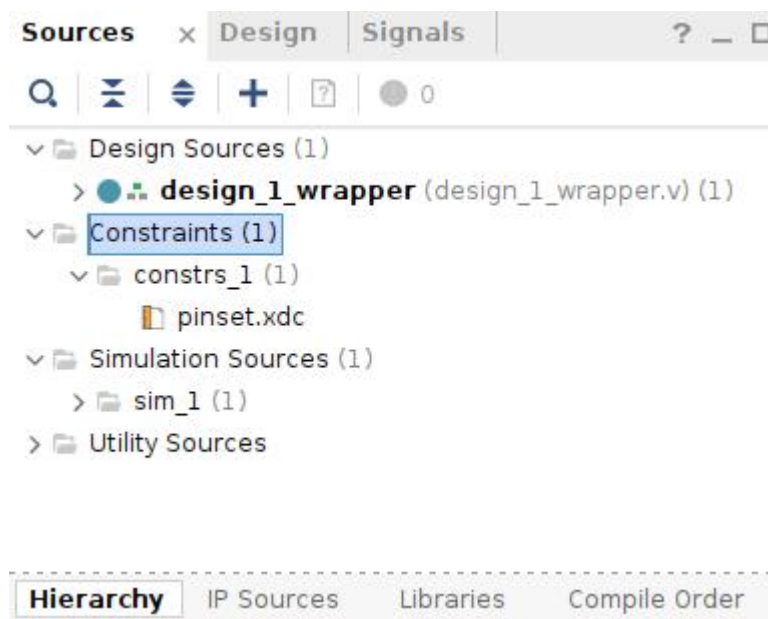
28) 不作任何改变，点击 “Next”，在如下界面中，点击按钮 “Create File”



29) 设置文件名称 “pinset”，点击 “OK ”。



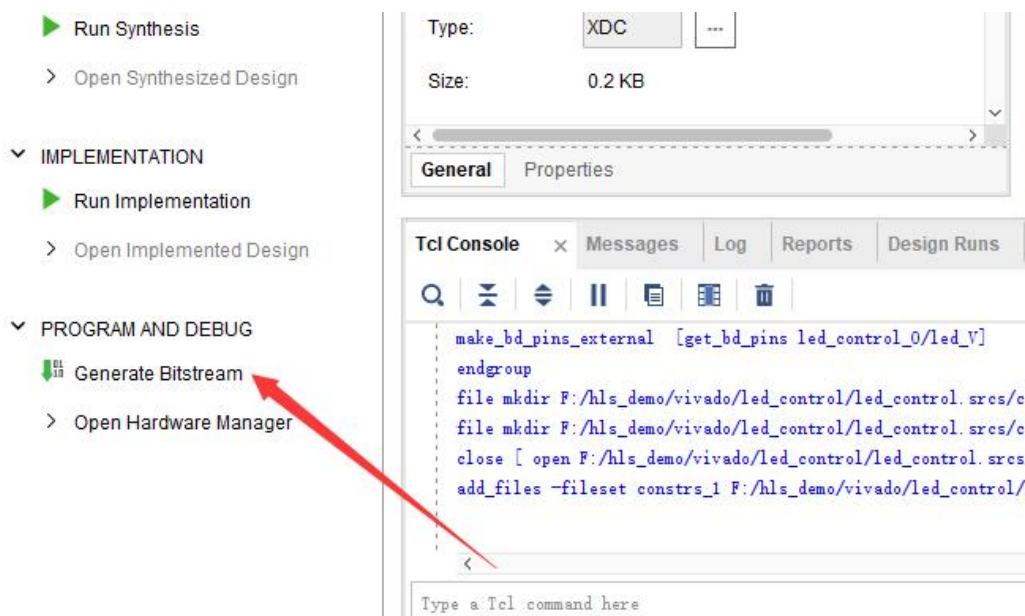
30) 回到前面的界面，点击“Finish”按钮，完成文件添加，此时我们展开“Constraints”，如下图。



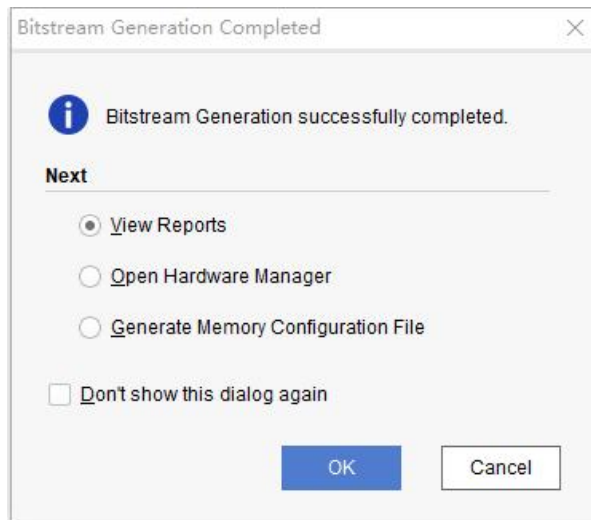
31) 双击“pinset.xdc”，在文件中输入如下内容，并“Ctrl+S”保存。注意：红色内容需要根据原理确定，必须替换。

```
set_property PACKAGE_PIN xxx [get_ports ap_clk_0]
set_property PACKAGE_PIN xxx [get_ports {led_V_0 [0]}]
set_property IOSTANDARD LVCMOSxx [get_ports {*}]
```

32) 至此，vivado 工程设计完毕，点击下述标注图标，生成 bit 文件。可能弹出保存文件对话框，点击“Save”即可。



33) 生成文件可能需要几分钟，直至弹出如下对话框，点击“Cancel”按钮，bit 文件生成完成。

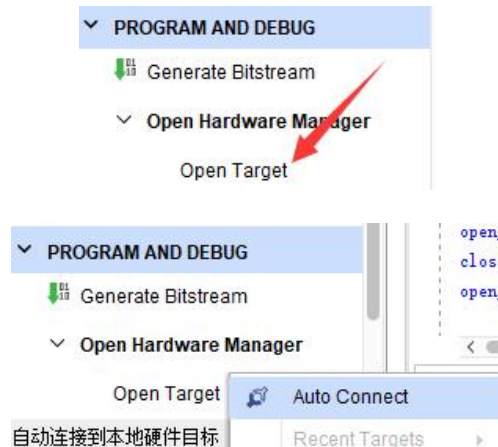


34) 接下来，可以加载刚生成的 bit 文件至板卡，先将板卡上电，并确定连接好 JTAG。

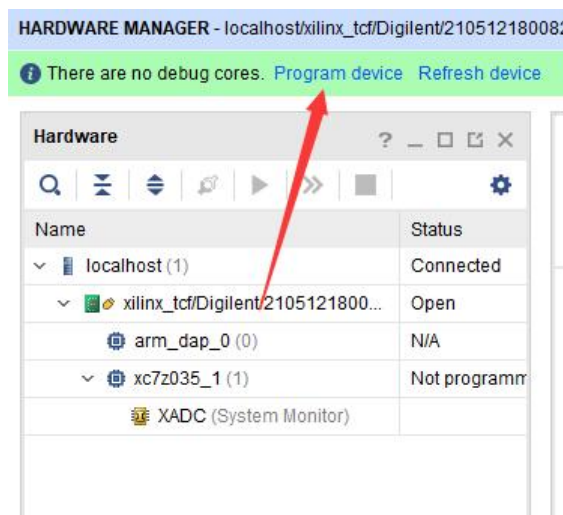
35) 点击 “Open Hardware Manager”



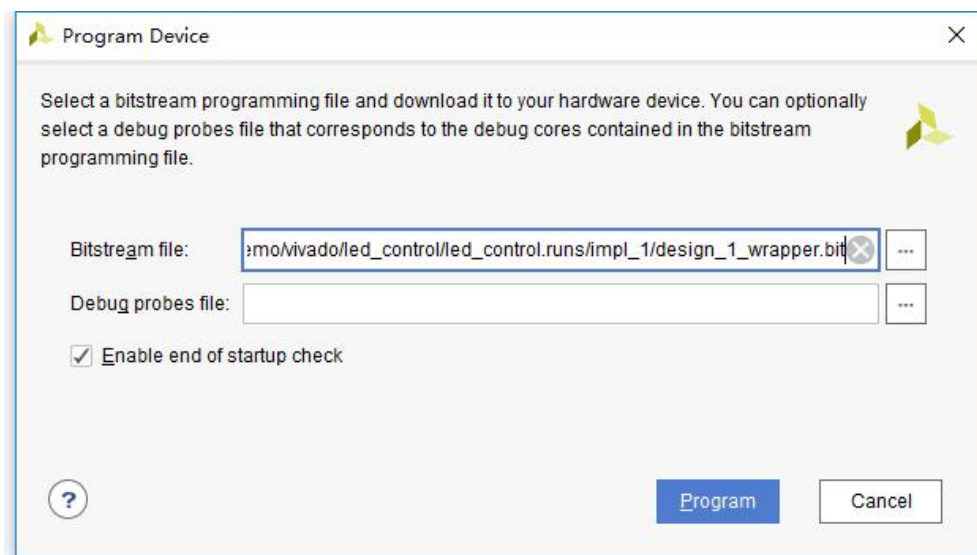
36) 点击“Open Target”→“Auto Connect”



37) 点击“Program device”



38) 点击 “Program”



39) 完成后，即可看到 led 灯以 1 秒的频率闪烁，至此，整个实验完成。

1.1.3 实验总结

通过 led 控制实验，我们学习了 HLS 与 vivado 的基本操作。对于 C、C++通过 HLS 转换为逻辑的流程，有了大致认识。

1.2 工程路径

名称	路径
Vivado 工程	vivado/led_control
HLS 工程	hls/led_control
bit 文件	/vivado/led_control/led_control.runs/impl_1/design_1_wrapper.bit

1.3 HLS 简介

Vitis hls 是 Xilinx 推出的高层次综合工具，可以实现直接使用 C, C++ 以及 System C 语言规范对赛灵思可编程器件进行编程，无需手动创建 RTL，从而可加速 IP 创建。

通过在 C, C++ 中插入 HLS Pragmas 语句，定义我们设计的 IP 与外部的接口，优化综合结果，如减少执行周期、减少 FPGA 资源使用等。插入方法：在 HLS 软件界面，点击程序所在的文件，在右侧边栏有个 Directive，里面列出了程序中所有用到的变量，函数和循环结构，点击右键通过向导插入语句。或者我们手动输入。

1.3.1 Vivado HLS 包含库

任意精度的数据类型	整数和定点 (ap_cint.h, ap_int.h and systemc.h)
HLS 流	流数据结构模型 旨在实现最佳性能和面积 (hls_stream.h)
HLS Math	广泛支持标准 C (math.h) 和 C++ (cmath.h) 数学库的综合。支持浮点和定点功能: abs, atan, atanf, atan2, atan2, ceil, ceilf, copysign, copysignf, cos, cosf, coshf, expf, fabs, fabsf, floorf, fmax, fmin, logf, fpclassify, isfinite, isinf, isnan, isnormal, log, log10, modf, modff, recip, recipf, round, rsqrt, rsqrtf, 1/sqrt, signbit, sin, sincos, sincosf, sinf, sinhf, sqrt, tan, tanf, trunc
HLS 视频	视频库可使用 C++ 实现多个方面的建模视频设计，支持 视频功能、特定数据类型、存储器线路缓存以及存储器视窗 (hls_video.h)。通过数据类型 hls::Mat, Vivado HLS 还与已有 OpenCV 功能兼容： AXIvideo2cvMat, AXIvideo2CvMat, AXIvideo2IplImage, cvMat2AXIvideo, CvMat2AXIvideo, cvMat2hlsMat, CvMat2hlsMat, CvMat2hlsWindow, hlsMat2cvMat, hlsMat2CvMat, hlsMat2IplImage, hlsWindow2CvMat, IplImage2AXIvideo, IplImage2hlsMat, AbsDiff, AddS, AddWeighted, And, Avg, AvgSdv, Cmp, CmpS, CornerHarris, CvtColor, Dilate, Duplicate, EqualizeHist, Erode, FASTX, Filter2D, GaussianBlur, Harris, HoughLines2, Integral, InitUndistortRectifyMap, Max, MaxS, Mean, Merge, Min, MinMaxLoc, MinS, Mul, Not, PaintMask, PyrDown, PyrUp, Range, Remap, Reduce, Resize, Set, Scale, Sobel, Split, SubRS, SubS, Sum, Threshold, Zero
HLS IP	集成 LogiCORE IP FFT 和 FIR Compiler (hls_fft.h, hls_fir.h, ap_shift_reg.h)
HLS 线性代数	cholesky, cholesky_inverse, matrix_multiply, qrf, qr_inverse, svd (hls_linear_algebra.h)
HLS DSP	atan2, awgn, cmpy, convolution_encoder, nco, qam_demod, qam_mod, sqrt, viterbi_decoder (hls_dsp.h)

1.3.2 Vivado HLS 接口

Argument Type	Scalar		Array			Pointer or Reference			HLS:: Stream
	Input	Return	I	I/O	O	I	I/O	O	
ap_ctrl_none									
ap_ctrl_hs		D							
ap_ctrl_chain									
axis									
s_axilite									
m_axi									
ap_none	D					D			
ap_stable									
ap_ack									
ap_vld								D	
ap_ovld							D		
ap_hs									
ap_memory			D	D	D				
bram									
ap_fifo									D
ap_bus									

 Supported
  D = Default Interface
  Not Supported

X14293

1.3.3 hls 官方教程

官方提供了两份很重要的教程：ug1399-vivado-high-level-synthesis.pdf。里面有非常详细的使用说明。

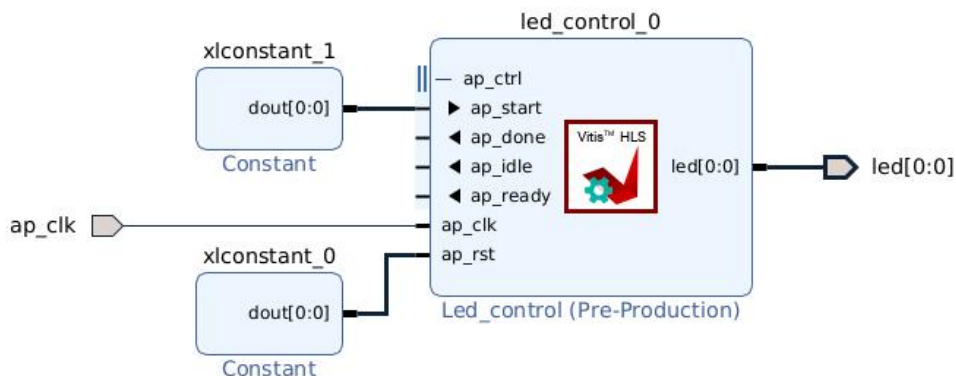
第二章 状态指示 led

2.1 模块控制 block-level

回顾前面章节我们做的 led 实验，源代码中插入了 “#pragma HLS interface ap_ctrl_none port=return ”,它的作用是定义该 IP 无控制端口。通常，一个模块可能需要被外部控制停止、开始。也需要向控制方表明，本模块当前的状态：空闲、运行中、已完成、就绪。这就需要有一个接口，hls 中，block-level I/O 协议为此定义了三种接口：ap_ctrl_none、ap_ctrl_hs 和 ap_ctrl_chain。ap_ctrl_none 无控制信号，ap_ctrl_hs 则多出 start、done、idle、ready 信号，ap_ctrl_chain 则在 ap_ctrl_hs 基础上多了 continue 信号。hls 中，ap_ctrl_hs 为该接口的默认值。在使用中，需要将 start 置高，如图：

我们可以看到，ap_ctrl_none 无控制信号，ap_ctrl_hs 则多出了 start、done、idle、ready 信号，ap_ctrl_chain 则在 ap_ctrl_hs 基础上多了 continue 信号。

hls 中，ap_ctrl_hs 为该接口的默认值。在使用中，需要将 start 置高，如图：



可以看出，选择默认 ap_ctrl_hs，使用起来比较麻烦，所以，之前的实验中，我们将其指定为 ap_ctrl_none。

2.2 可配置模块

若 led 的闪烁频率可通过软件配置，就可以将其作为状态指示灯了。为此，我们修改 hls 代码如下：

```
#include <ap_int.h>

void led_register(ap_int<1> &led, int total_cnt, int high_cnt)
{
    #pragma HLS INTERFACE ap_none port=led
}
```

```
#pragma HLS INTERFACE s_axilite port=total_cnt
#pragma HLS INTERFACE s_axilite port=high_cnt
#pragma HLS INTERFACE ap_ctrl_none port=return
    led = 0;
    for(int i=0;i<total_cnt;i++)
    {
#pragma HLS LOOP_TRIPCOUNT avg=100000000 max=100000000 min=100000000
        if(i == high_cnt)
            led = ~led;
    }
}
```

这里，我们新增了两个参数 `total_cnt`、`high_cnt`，并将其指定为 `s_axilite` 接口，此时软件可通过 AXI4-Lite 接口来配置这两个参数值。

2.3 工程路径

名称	路径
Vivado 工程	vivado/led_register
HLS 工程	hls/led_register
BOOT.bin 文件	bootimage

2.4 实验结果

程序运行后，可以看到 led 灯每秒闪烁一次。

第三章 视频彩条

3.1 Vivado HLS 视频开发

3.1.1 与 OpenCV 关系

开源计算机视觉 (OpenCV) 被广泛用于开发计算机视觉应用, 它包含 2500 多个优化的视频函数的函数库并且专门针对台式机处理器和 GPU 进行优化。OpenCV 的用户成千上万, OpenCV 的设计无需修改即可在 Zynq 器件的 ARM 处理器上运行。但是利用 OpenCV 实现的高清处理经常受外部存储器的限制, 尤其是存储带宽会成为性能瓶颈, 存储访问也会限制功耗效率。使用 VivadoHLS 高级语言综合工具, 可以轻松实现 OpenCV C++ 视频处理设计到 RTL 代码的转换, 输出硬件加速器或者直接在 FPGA 上实现实时视频处理功能。同时, Zynq All-programmable SOC 是实现嵌入式计算机视觉应用的极好方法, 很好解决了在单一处理器上实现视频处理性能低功耗高的限制。

OpenCV 图像处理是基于存储器帧缓存而构建的, 它总是假设视频 frame 数据存放在外部 DDR 存储器中, 因此, OpenCV 对于访问局部图像性能较差, 因为处理器的小容量高速缓存性能不足以完成这个任务。而且出于性能考虑, 基于 OpenCV 设计的架构比较复杂, 功耗更高。在对分辨率或帧速率要求低, 或者在更大的图像中对需要的特征或区域进行处理是, OpenCV 似乎足以满足很多应用的要求, 但对于高分辨率高帧率实时处理的场景下, OpenCV 很难满足高性能和低功耗的需求。

基于视频流的架构能提供高性能和低功耗, 链条化的图像处理函数能减少外部存储器访问, 针对视频优化的行缓存和窗口缓存比处理器高速缓存更简单, 更易于用 FPGA 部件, 使用 VivadoHLS 中的数据流优化来实现。

VivadoHLS 对 OpenCV 的支持, 不是指可以将 OpenCV 的函数库直接综合成 RTL 代码, 而是需要将代码转换为可综合的代码, 这些可综合的视频库称为 HLS 视频库, 由 VivadoHLS 提供。

OpenCV 函数不能直接通过 HLS 进行综合, 因为 OpenCV 函数一般都包含动态的内存分配、浮点以及假设图像在外部存储器中存放或者修改。

VivadoHLS 视频库用于替换很多基本的 OpenCV 函数, 它与 OpenCV 具有相似的接口和算法, 主要针对在 FPGA 架构中实现的图像处理函数, 包含了专门面向 FPGA 的优化, 比如定点运算而非浮点运算 (不必精确到比特位), 片上的行缓存 (line buffer) 和窗口缓存 (window buffer)。

3.1.2 VivadoHLS 视频库函数

HLS 视频库是包含在 hls 命名空间内的 C++ 代码。使用时需要包含头文件 "hls_video.h"。

该库与 OpenCV 等具有相似的接口和等效的行为, 例如:

OpenCV 库: `cvScale(src, dst, scale, shift);`

HLS 视频库: `hls::Scale<...>(src, dst, scale, shift);`

一些构造函数具有类似的或替代性的模板参数, 例如:

OpenCV 库: `cv::Mat mat(rows, cols, CV_8UC3);`

HLS 视频库: `hls::Mat mat(rows, cols);`

3.2 实验介绍

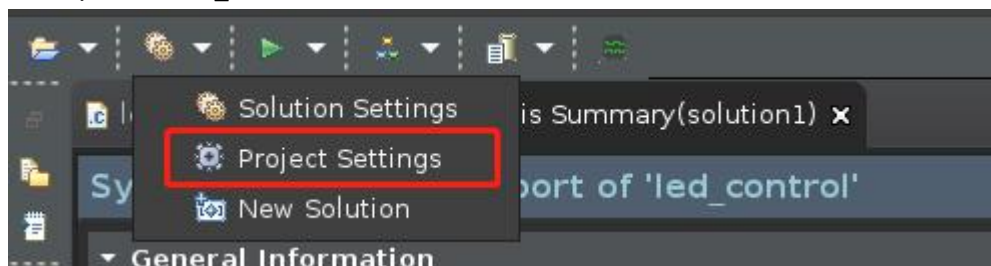
在图像处理中, 我们常用彩条(colorbar)来验证视频输出接口, 或作为无视频时的输出方案。我们这里的 colorbar, 为 8 个不同颜色的竖条, 中间三分之一的竖条, 从左至右滚动, 其它区域静止。且输出的图像, 可以动态修改分辨率。

3.3 HLS IP 创建

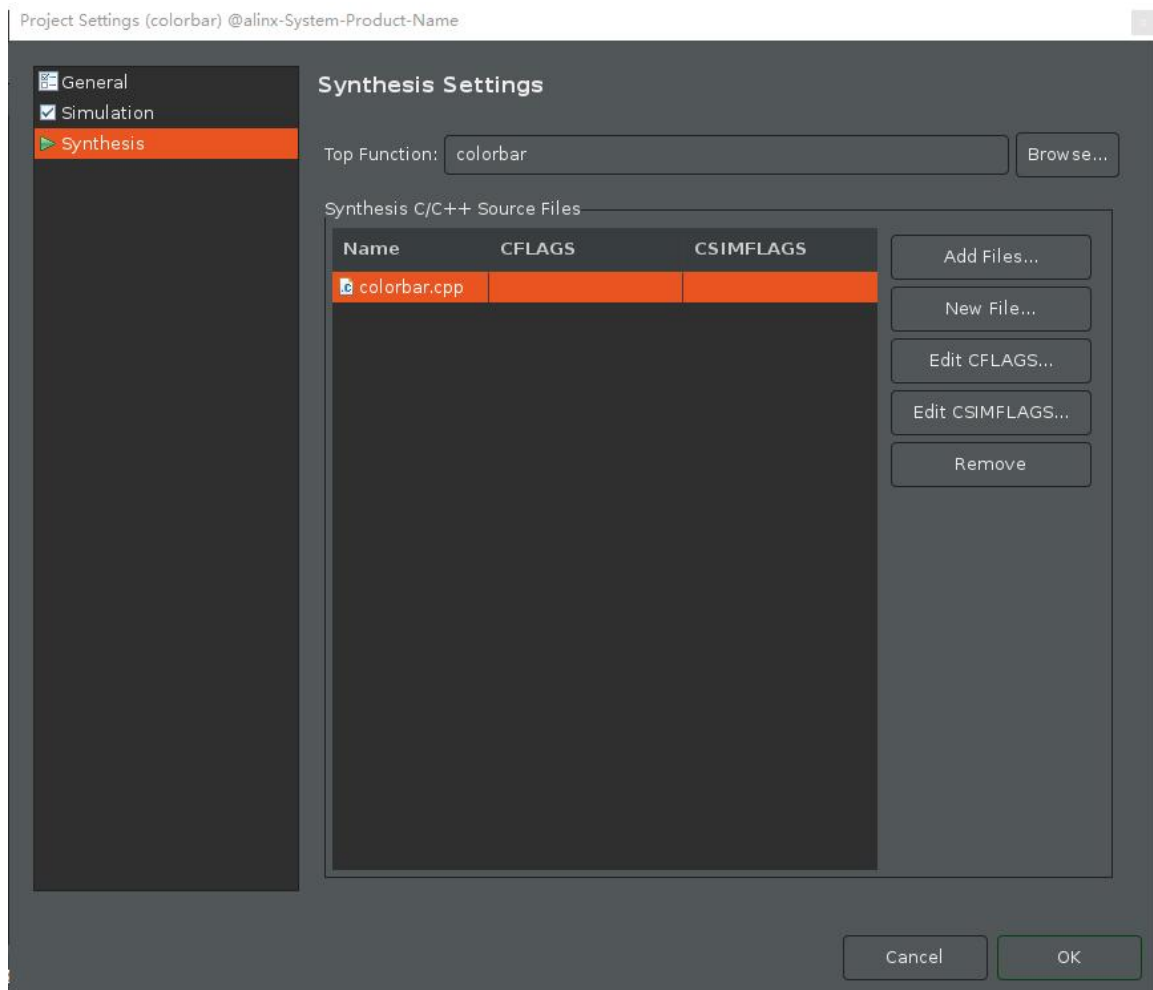
3.3.1 前期准备

自从 2020.1 版本开始, vitis_hls 对于图像处理常用的算法函数需要额外从赛灵思官方 github 上下载, 网址为: https://github.com/Xilinx/Vitis_Libraries 下载完成之后解压, 如果要用到其中的函数我们按照如下步骤进行操作

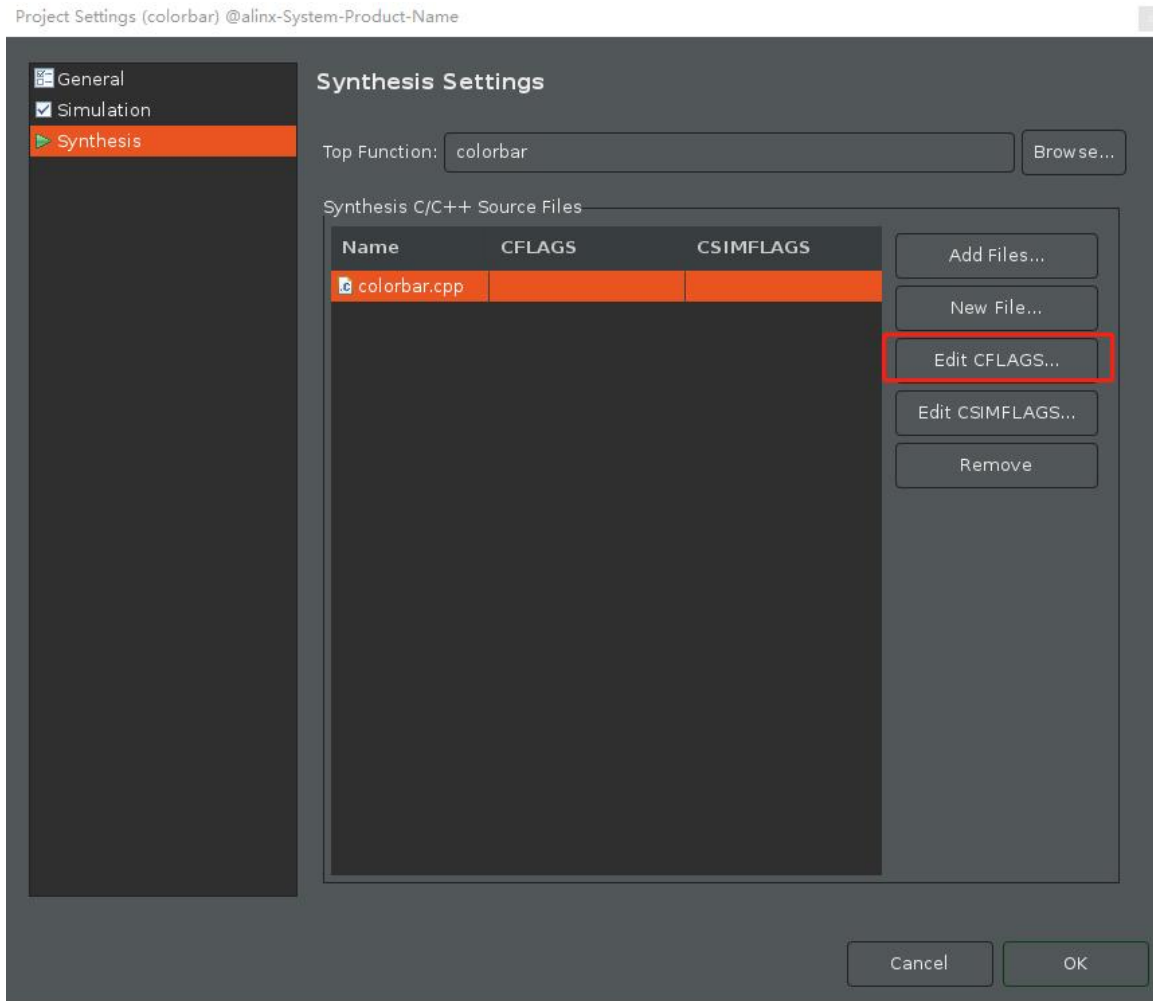
1) 打开 vitis_hls ,点击图标



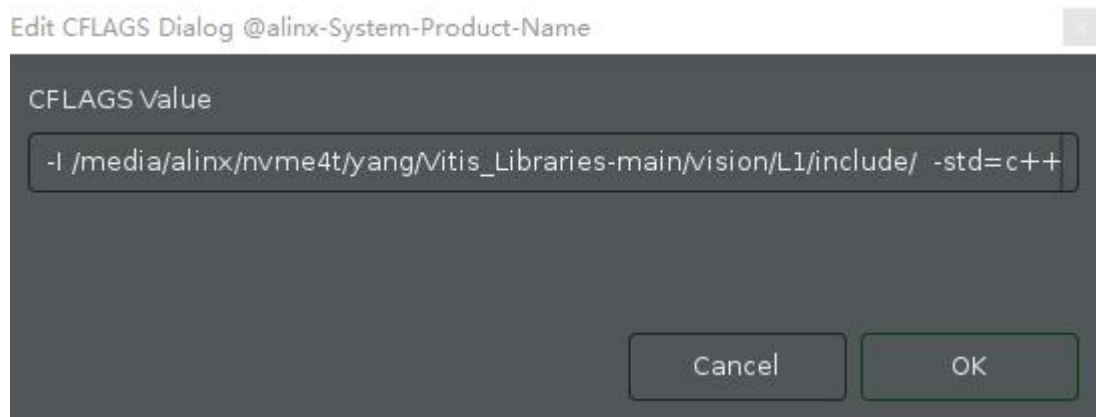
2) 选择综合



3) 单击 edit CFLAGS



4) 输入-I+安装的 vitis_libraries 文件夹路径+std=c++0x,这里输入的是绝对路径
单击 OK, 接下来就可以调用需要的库函数对图像进行相应的处理



3.3.2 源代码

```
#include "colorbar.h"

template<int ROWS, int COLS>
void createColorBar(xf::cv::Mat<XF_8UC3, ROWS, COLS, XF_NPPC1>
&imgColorbar)
```

```
{
    XF_TNAME(XF_8UC3,XF_NPPC1) pixel;
    int move, wBar;
    int rows = imgColorbar.rows;
    int cols = imgColorbar.cols;
    static int moving = 0;

    for(int row=0;row<rows;row++)
    {
#pragma HLS loop_flatten off
        if( (row < rows/3) || (row >= rows/3*2) )
        {
            move = 0;
        }
        else
        {
            move = moving;
        }
        wBar = cols>>3;
        for(int col=0;col<cols;col++)
        {
#pragma HLS pipeline
            int index = ((col+move)/wBar)&0x7;
            switch(index)
            {
                case 0:
                    pixel.range(7,0) = 0xff;
                    pixel.range(15,8) = 0xff;
                    pixel.range(23,16) = 0xff; //white
                    break;
                case 1:
                    pixel.range(7,0) = 0x00;
                    pixel.range(15,8) = 0xff;
                    pixel.range(23,16) = 0xff; //yellow
                    break;
                case 2:
                    pixel.range(7,0) = 0xff;
                    pixel.range(15,8) = 0xff;
                    pixel.range(23,16) = 0x00; //cyan
                    break;
                case 3:
                    pixel.range(7,0) = 0x00;
                    pixel.range(15,8) = 0xff;
                    pixel.range(23,16) = 0x00; //green
                    break;
                case 4:
                    pixel.range(7,0) = 0xff;
                    pixel.range(15,8) = 0x00;
```

```
        pixel.range(23,16) = 0xff; //magenta
        break;
    case 5:
        pixel.range(7,0) = 0x0;
        pixel.range(15,8) = 0x0;
        pixel.range(23,16) = 0xff; //red
        break;
    case 6:
        pixel.range(7,0) = 0xff;
        pixel.range(15,8) = 0x00;
        pixel.range(23,16) = 0x00; //blue
        break;
    default:
        pixel.range(7,0) = 0x00;
        pixel.range(15,8) = 0x00;
        pixel.range(23,16) = 0x00; //black
        break;
    }

    imgColorbar.write(row*cols+col,pixel);
}
}
if(!moving)
{
    moving = cols;
}
else
{
    moving--;
}
}

void colorbar(pixel_stream &dst, int rows, int cols)
{
    #pragma HLS INTERFACE axis port=dst
    #pragma HLS INTERFACE s_axilite port=rows
    #pragma HLS INTERFACE s_axilite port=cols
    #pragma HLS INTERFACE s_axilite port=return
    #pragma HLS INTERFACE ap_ctrl_none port=return
    #pragma HLS dataflow
    xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS,IMG_MAX_COLS,XF_NPPC1>imgColorbar;
    createColorBar<IMG_MAX_ROWS,IMG_MAX_COLS>(imgColorbar);
    xf::cv::xfMat2AXIvideo(imgColorbar, dst);
}
```

3.3.3 接口介绍

数据类型 `pixel_stream` 的定义为 `hls::stream< ap_axiu<24,1,1,1> >`。这是 HLS 的流接口，其中 24 指 RGB 数据位宽共 24Bit，其它位为流控制信号。通常，各模块之间视频数据接口都使用流接口。这里，我们将“dst”定义为视频输出流。

“rows”与“cols”分别定义输出图像的高与宽，“mode”用于表明当前是否处于配置状态，若处于配置状态，则不输出图像。这里，我们使用 `axi-lite` 来管理这些接口，包括模块的控制接口。

3.3.4 hls::Mat 介绍

OpenCV 中常见的与图像操作有关的数据容器有 `Mat`，`CvMat` 和 `IplImage`，这三种类型都可以代表和显示图像，但是，`Mat` 类型侧重于计算，数学性较高，`openCV` 对 `Mat` 类型的计算也进行了优化。而 `CvMat` 和 `IplImage` 类型更侧重于“图像”，`opencv` 对其中的图像操作（缩放、单通道提取、图像阈值操作等）进行了优化。`Mat` 类型较 `CvMat` 与 `IplImage` 类型来说，有更强的矩阵运算能力，支持常见的矩阵运算。在计算密集型应用当中，将 `CvMat` 与 `IplImage` 类型转化为 `Mat` 类型将大大减少计算时间花费。

Vitis HLS 视频处理函数库使用 `xf::cv::Mat<>` 数据类型，这种类型用于模型化视频像素流处理，在 HLS 实现 OpenCV 的设计中，需要将输入和输出 HLS 可综合的视频设计接口，修改为 `Video stream` 接口，也就是采用 HLS 提供的 `video` 接口可综合函数，实现 AXI4 `video stream` 到 Vivado HLS 中 `hls::Mat<>` 类型间的转换。

在本例程中，我们在顶层函数中定义了 `xf::cv::Mat<XF_8U3,1080, 1920, XF_NPPC1>` `imgColorbar`。其中，前面的尖括号内分别表示我们的图像最大支持分辨率为 1920x1080，bit24 的图像，后面括号参数表示当前实际的高与宽分别为 `rows`、`cols`。

3.3.5 优化

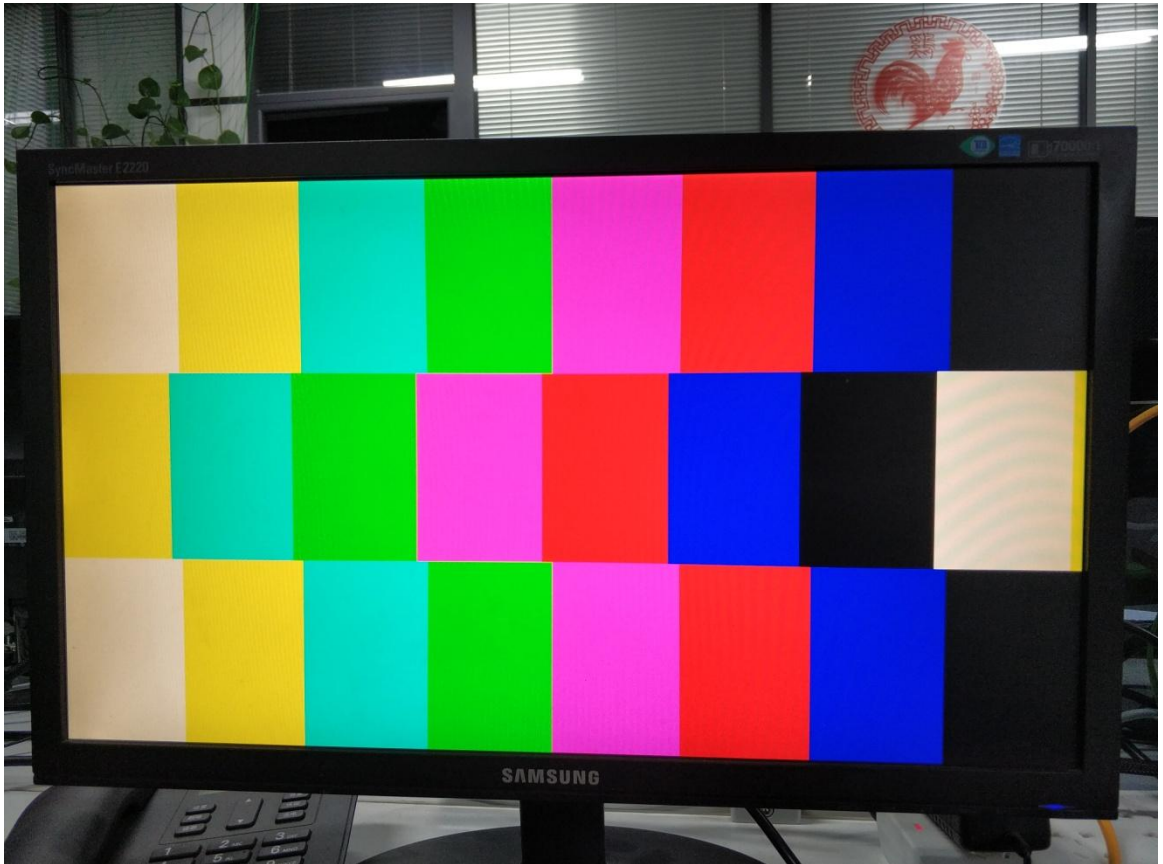
在顶层函数 `colorbar` 中，我们使用了语句“`#pragma HLS dataflow`”，以使能够在下面的函数中，形成流水操作，提高数据处理效率。

在 `createColorBar` 中，在第一个 `for` 循环内嵌入“`#pragma HLS loop_flatten off`”，表明放弃与内部 `for` 循环整合，这样会增加时钟周期，但改善了信号延迟。

3.4 工程路径

名称	路径
Vivado 工程	vivado/colorbar
HLS 工程	hls/colorbar
BOOT.bin 文件	bootimage

3.5 实验结果



显示一个 colorbar，中间部分滚动显示。

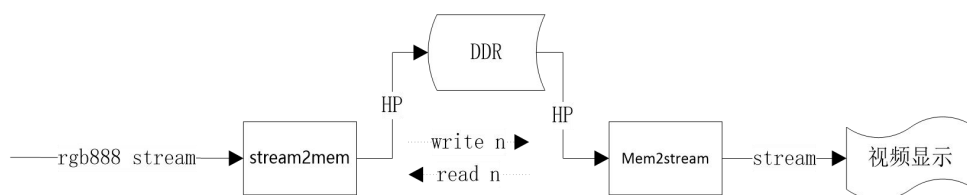
需要注意，显示输出分辨率在不断变化，所以画面会隔一段时间黑掉，属于正常现象。

第四章 视频帧缓存读写管理

4.1 实验介绍

首先，将镜头的图像(转换为 RGB888)存储至 DDR，然后通过读取 DDR 来显示视频图像。这里需要解决图像刷新速率不一致时，可能造成的图像割裂状问题。

这里共设计了二个 HLS 模块：stream2mem 模块，mem2stream 模块，流程图如下。



这里需要解决的一个问题是，若写 DDR 与读 DDR 速度不匹配，则会造成图像显示时裂开的现象。解决的方法是：DDR 使用三帧缓存，HLS 读模块与写模块分别向对方反馈帧索引号。

4.2 模块主要代码

```
mem2stream 部分
#include "mem2stream.h"
unsigned int cache_len=0;

void readmem(unsigned int *pMemPort, unsigned int base, int row, int rows,
unsigned int *to, int line_len)
{
    if(cache_len < (2*line_len))
    {
        if((row+1) == rows)
        {
            memcpy(to, pMemPort+base, (2*line_len-cache_len)*4);
        }
        else
        {
            memcpy(to, pMemPort+base+(row+1)*line_len,
(2*line_len-cache_len)*4);
        }
        cache_len = line_len;
    }
    else
    {
        cache_len -= line_len;
    }
}
```

```

template<int ROWS, int COLS>
void mem2mat(xf::cv::Mat<XF_8UC3,ROWS,COLS,XF_NPPC1> &img, unsigned int
*pMemPort,int baseAddr[3],int w, int &r)
{
    XF_TNAME(XF_8UC3,XF_NPPC1) pixel;
    unsigned int cacheBuff[COLS];
    int local_rows = img.rows;
    int local_cols = img.cols;
    int line_len    = img.cols*3/4;
    static int index=2;
    int n = (index<2)?(index+1):0;
    index = (n == w)?index:n;
    r = index;
    #pragma HLS STREAM variable=cacheBuff depth=COLS*2
    for(int row=0; row<local_rows; row++)
    {
        #pragma HLS loop_tripcount min=12 max=1080
        #pragma HLS dataflow
        readmem(pMemPort, baseAddr[index]>>2, row, local_rows, cacheBuff,
line_len);
        for(int col=0; col<local_cols; col+=4)
        {
            #pragma HLS loop_tripcount min=3 max=480
            pixel.range(7,0) = cacheBuff[0]>>0;
            pixel.range(15,8) = cacheBuff[0]>>8;
            pixel.range(23,16) = cacheBuff[0]>>16;
            img.write(row*local_cols+col,pixel);
            pixel.range(7,0) = cacheBuff[0]>>24;
            pixel.range(15,8) = cacheBuff[1]>>0;
            pixel.range(23,16) = cacheBuff[1]>>8;
            img.write(row*local_cols+col+1,pixel);
            pixel.range(7,0) = cacheBuff[1]>>16;
            pixel.range(15,8) = cacheBuff[1]>>24;
            pixel.range(23,16) = cacheBuff[2]>>0;
            img.write(row*local_cols+col+2,pixel);
            pixel.range(7,0) = cacheBuff[2]>>8;
            pixel.range(15,8) = cacheBuff[2]>>16;
            pixel.range(23,16) = cacheBuff[2]>>24;
            img.write(row*local_cols+col+3,pixel);
        }
    }
}

void mem2stream(vstream_t &vstream, unsigned int *pMemPort, int rows, int
cols,int baseAddr[3],int indexw, int &indexr)
{
    #pragma HLS INTERFACE mode=s_axilite port=baseAddr

```

```

#pragma HLS INTERFACE axis port=vstream
#pragma HLS INTERFACE m_axi port=pMemPort
#pragma HLS INTERFACE s_axilite port=rows
#pragma HLS INTERFACE s_axilite port=cols
#pragma HLS INTERFACE s_axilite port=return
#pragma HLS INTERFACE ap_none port=indexw
#pragma HLS INTERFACE ap_none port=indexr
    xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS,IMG_MAX_COLS,XF_NPPC1> img;
#pragma HLS STREAM depth=1080 type=fifo variable=img

#pragma HLS dataflow
    mem2mat<IMG_MAX_ROWS,IMG_MAX_COLS>(img,
pMemPort,baseAddr,indexw,indexr);
    xf::cv::xfMat2AXIvideo(img, vstream);
}

stream2mem 部分
#include "stream2mem.h"
void writemem(unsigned int *pMemPort, unsigned int to, unsigned int *from,
int len)
{
    if(len > 0)
    {
        memcpy(pMemPort+to, from, len);
    }
}

template<int ROWS, int COLS>
void mat2mem(xf::cv::Mat<XF_8UC3,ROWS,COLS,XF_NPPC1> &img, unsigned int
*pMemPort,int baseAddr[3],int &w, int r)
{
    XF_TNAME(XF_8UC3,XF_NPPC1) pixelA, pixelB, pixelC, pixelD;
    unsigned int cacheBuff[COLS*3/4];
    int local_rows = img.rows;
    int local_cols = img.cols;
    int line_len = img.cols*3/4;
    static int index=0;
    int n = (index<2)?(index+1):0;
    index = (n == r)?index:n;
    w = index;
#pragma HLS STREAM variable=cacheBuff depth=COLS/4
    for(int row=0; row<local_rows; row++)
    {
#pragma HLS loop_tripcount min=12 max=1080
#pragma HLS dataflow
        for(int col=0; col<local_cols; col+=4)
        {
#pragma HLS loop_tripcount min=3 max=480

```

```

        pixelA = img.read(row*local_cols+col);
        pixelB = img.read(row*local_cols+col+1);
        cacheBuff[0] =
(pixelA.range(7,0)<<0)|(pixelA.range(15,8)<<8)|(pixelA.range(23,16)<<16)|
(pixelB.range(7,0)<<24);
        pixelC = img.read(row*local_cols+col+2);
        cacheBuff[1] =
(pixelB.range(15,8)<<0)|(pixelB.range(23,16)<<8)|(pixelC.range(7,0)<<16)|
(pixelC.range(15,8)<<24);
        pixelD = img.read(row*local_cols+col+3);
        cacheBuff[2] =
(pixelC.range(23,16)<<0)|(pixelD.range(7,0)<<8)|(pixelD.range(15,8)<<16)|
(pixelD.range(23,16)<<24);
    }
    writemem(pMemPort, (baseAddr[index]>>2)+(row*line_len),
cacheBuff, line_len*4);
    }
}

void stream2mem(vstream_t &vstream, unsigned int *pMemPort, int rows, int
cols, int baseAddr[3],int &indexw, int indexr)
{
#pragma HLS INTERFACE mode=s_axilite port=baseAddr
#pragma HLS INTERFACE axis port=vstream
#pragma HLS INTERFACE m_axi port=pMemPort
#pragma HLS INTERFACE s_axilite port=rows
#pragma HLS INTERFACE s_axilite port=cols
#pragma HLS INTERFACE ap_ctrl_none port=return
#pragma HLS INTERFACE ap_none port=indexw
#pragma HLS INTERFACE ap_none port=indexr

    xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS,IMG_MAX_COLS,XF_NPPC1> img;
#pragma HLS STREAM depth=1920 type=pipo variable=img

#pragma HLS dataflow
    xf::cv::AXIvideo2xfMat(vstream, img);
    mat2mem(img, pMemPort,baseAddr,indexw, indexr);
}
}
}

```

在这里，我们将数组 `cacheBuff` 定义为流类型，方便我们排流水。另外，`baseAddr` 为全局变量，但我们仍然可以将其定义为模块的外部接口，而不必是 `top` 函数的参数。

内存读取与写入时，调用系统函数 `memcpy`。若直接引用单个数组成员，将严重浪费 DDR 带宽，不推荐。

4.1 工程路径

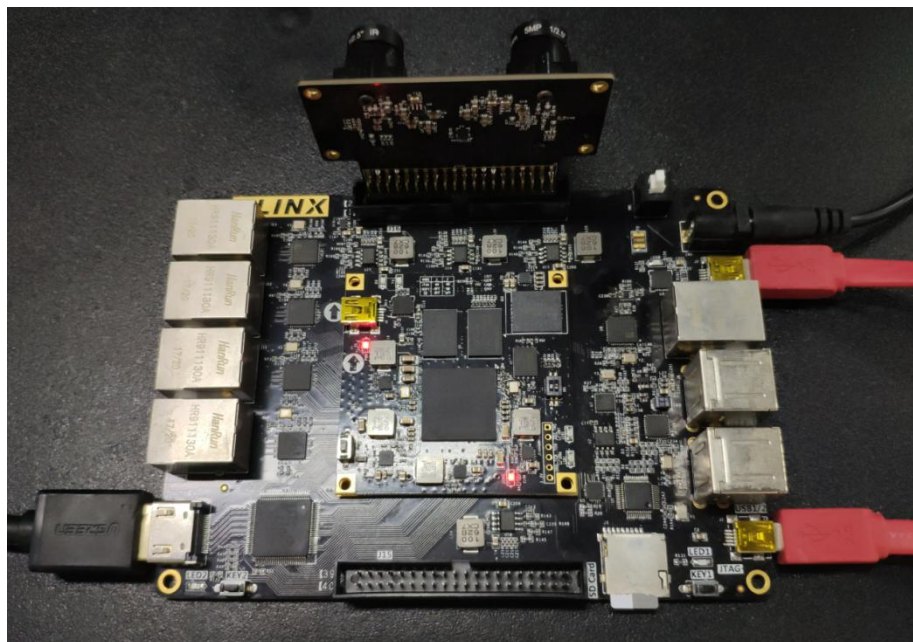
名称	路径
Vivado 工程	vivado/video_show
HLS 工程	hls/mem2stream
HLS 工程	hls/stream2mem
BOOT.bin 文件	bootimage

4.2 实验结果

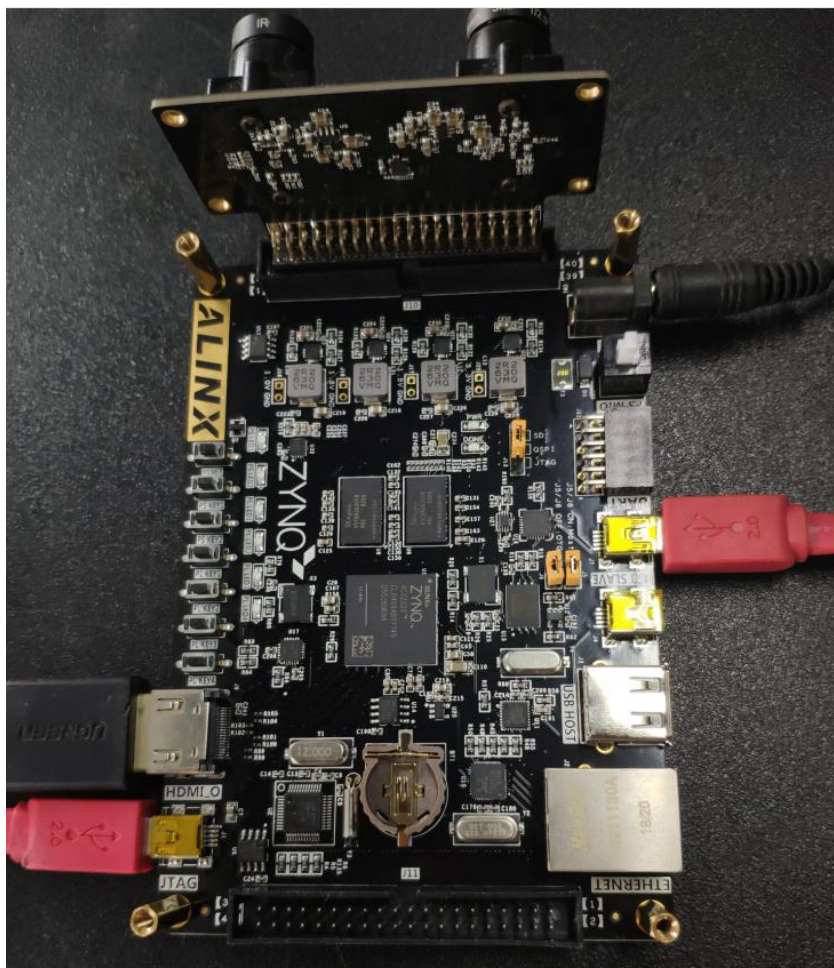


显示双目视频中的一个视频图像。当视频中出现快速移动物体时，不会有割裂状。当前视频在几个分辨率之间不断切换。低分辨率时，仅能显示部分图像。

需要注意，显示输出分辨率在不断变化，所以画面会隔一段时间黑掉，属于正常现象。



AX7021 硬件连接图 (J16 扩展口)



AX7020 硬件连接图(J10 扩展口)

第五章 图像对比度调整

5.1 实验介绍

直方图均衡技术将原始图像的灰度直方图从比较集中的某个灰度区间变成在全部灰度范围内的均匀分布，是一种常用的图像增强方法。

首先, 我们需要将 RGB888 图像转化为 YCrCb, 统计当前 Y 分量直方图分布 (应用于下一帧), 并通过上一帧的统计数据均衡化当前 Y 分量, 再转换回 RGB888 并输出。

5.2 模块主要代码

```
#include "contrastadj.h"

void ExtractPixel(XF_TNAME(XF_8UC3, XF_NPPC1) &src_axi, ap_uint<8>
dst[3])
{
    unsigned int i, j=0;
    for(i=0; i<24; i+=8)
    {
        #pragma HLS PIPELINE
        dst[j]=src_axi.range(i+7, i);
        j++;
    }
}

void PackPixel(XF_TNAME(XF_8UC3, XF_NPPC1)&dst_axi, ap_uint<8>src[3])
{
    unsigned int i, j=0;
    for(i=0; i<24; i+=8)
    {
        #pragma HLS PIPELINE
        dst_axi.range(i+7, i)=src[j];
        j++;
    }
}

template<int ROWS, int COLS, int N>
void dualAryEqualize(
    xf::cv::Mat<XF_8UC3, ROWS, COLS, XF_NPPC1>    &_src,
    xf::cv::Mat<XF_8UC3, ROWS, COLS, XF_NPPC1>    &_dst,
    int filter)
{
    static ap_uint<8> map[N];
    ap_uint<32>hist_out1[N];
```

```

#pragma HLS ARRAY_PARTITION variable=hist_out1 dim=1 complete
ap_uint<32> hist_out2[N];
#pragma HLS ARRAY_PARTITION variable=hist_out2 dim=1 complete
unsigned int i,j=0;
XF_TNAME(XF_8UC3,XF_NPPC1)src1;
XF_TNAME(XF_8UC3,XF_NPPC1)src2;
XF_TNAME(XF_8UC3,XF_NPPC1)dst1;
XF_TNAME(XF_8UC3,XF_NPPC1)dst2;
ap_uint<8> value1[3];
ap_uint<8> value2[3];
int pos1,pos2;
for(i=0;i<N;i++)
{
    hist_out1[i]=0;
    hist_out2[i]=0;
}

for(i=0;i<ROWS;i++)
{
    for(j=0;j<COLS;j+=2)
    {
#pragma HLS DEPENDENCE variable=src1 intra false
#pragma HLS DEPENDENCE variable=src2 intra false

#pragma HLS PIPELINE II=2
        src1=_src.read(i*COLS+j);
        src2=_src.read(i*COLS+j+1);
        ExtractPixel(src1,value1);
        ExtractPixel(src2,value2);
        pos1=(int)value1[0];
        pos2=(int)value2[0];
        hist_out1[pos1]+=1;
        hist_out2[pos2]+=1;
        value1[0]=map[pos1];
        value2[0]=map[pos2];
        PackPixel(dst1,value1);
        PackPixel(dst2,value2);
        _dst.write(i*COLS+j,dst1);
        _dst.write(i*COLS+j+1,dst2);
    }
}
int count_total = 0, tmp_hist;
float scale;
for(int i=0;i<N;i++)
{
#pragma HLS PIPELINE
    tmp_hist=hist_out1[i]+hist_out2[i];

```



```

        count_total += tmp_hist;
        scale = count_total*255/ROWS/COLS;
        scale=(scale-i)*0.01*filter+i;
        if(scale < 16)scale = 16;
        if(scale > 255)scale = 235;
        map[i] = ap_uint<8> (scale);
    }
}

template <int ROWS, int COLS>
void xfrgb2ycrcb(xf::cv::Mat<XF_8UC3, ROWS, COLS,XF_NPPC1>& src,
                xf::cv::Mat<XF_8UC3, ROWS, COLS,XF_NPPC1>& dst)
{
    XF_TNAME(XF_8UC3,XF_NPPC1)rgb_packed;
    XF_TNAME(XF_8UC3,XF_NPPC1)ycrcb_packed;
    ap_uint<8>rgb[3];
    ap_uint<8>ycrcb[3];
    unsigned int i,j=0;
    for(i=0;i<ROWS;i++)
    {
        for(j=0;j<COLS;j++)
        {
            #pragma HLS PIPELINE
                rgb_packed=src.read(i*COLS+j);
                ExtractPixel(rgb_packed,rgb);
                ycrcb[0]=CalculateGRAY(rgb[0],rgb[1],rgb[2]);
                ycrcb[1]=Calculate_CR(rgb[0],ycrcb[0]);
                ycrcb[2]=Calculate_CB(rgb[2],ycrcb[0]);
                PackPixel(ycrcb_packed,ycrcb);
                dst.write(i*COLS+j,ycrcb_packed);
        }
    }
}

template<int ROWS,int COLS>
void xfycrcb2rgb(xf::cv::Mat<XF_8UC3, ROWS, COLS,XF_NPPC1>& src,
                xf::cv::Mat<XF_8UC3, ROWS, COLS,XF_NPPC1>& dst)
{
    XF_TNAME(XF_8UC3,XF_NPPC1)rgb_packed;
    XF_TNAME(XF_8UC3,XF_NPPC1)ycrcb_packed;
    ap_uint<8>rgb[3];
    ap_uint<8>ycrcb[3];
    unsigned int i,j=0;
    for(i=0;i<ROWS;i++)
    {
        for(j=0;j<COLS;j++)
        {
            #pragma HLS PIPELINE

```

```

        ycrb_packed=src.read(i*COLS+j);
        ExtractPixel(ycrb_packed,ycrb);
        rgb[0]=Calculate_Ycrb2R(ycrb[0],ycrb[1]);
        rgb[1]=Calculate_Ycrb2G(ycrb[0],ycrb[1],ycrb[2]);
        rgb[2]=Calculate_Ycrb2B(ycrb[0],ycrb[2]);
        PackPixel(rgb_packed,rgb);
        dst.write(i*COLS+j,rgb_packed);
    }
}
}

void contrastadj(hls::stream<ap_axiu<24,1,1,1>>
&src_axi,hls::stream<ap_axiu<24,1,1,1>> &dst_axi,int adj)
{

#pragma HLS INTERFACE mode=s_axilite port=return
#pragma HLS INTERFACE mode=s_axilite port=adj register

#pragma HLS INTERFACE axis port=src_axi
#pragma HLS INTERFACE axis port=dst_axi

    xf::cv::Mat<XF_8UC3,IMG_MAX_HEIGHT, IMG_MAX_WIDTH, XF_NPPC1> img1;
#pragma HLS STREAM depth=1920 type=fifo variable=img1
    xf::cv::Mat<XF_8UC3,IMG_MAX_HEIGHT, IMG_MAX_WIDTH, XF_NPPC1> img2;
#pragma HLS STREAM depth=1920 type=fifo variable=img2
    xf::cv::Mat<XF_8UC3,IMG_MAX_HEIGHT, IMG_MAX_WIDTH, XF_NPPC1> img3;
#pragma HLS STREAM depth=1920 type=fifo variable=img3
    xf::cv::Mat<XF_8UC3,IMG_MAX_HEIGHT, IMG_MAX_WIDTH, XF_NPPC1> img4;
#pragma HLS STREAM depth=1920 type=fifo variable=img4

#pragma HLS dataflow

    // AXIvideoTest<IMG_MAX_HEIGHT,IMG_MAX_WIDTH>(src_axi,dst_axi);
    xf::cv::AXIvideo2xfMat<24,XF_8UC3,IMG_MAX_HEIGHT,IMG_MAX_WIDTH,XF_NPPC1>(src_axi, img1);
    xfrgb2ycrb<IMG_MAX_HEIGHT,IMG_MAX_WIDTH>(img1, img2);
    dualAryEqualize<IMG_MAX_HEIGHT, IMG_MAX_WIDTH, 256>(img2, img3,adj);
    xfycrb2rgb<IMG_MAX_HEIGHT,IMG_MAX_WIDTH>(img3, img4);
    xf::cv::xfMat2AXIvideo<24,XF_8UC3,IMG_MAX_HEIGHT,IMG_MAX_WIDTH,XF_NPPC1>(img4,dst_axi);
}

```

这里进行数据统计的时候，我们一次处理了两个像素点。因为在一个时钟内，我们无法完成“hist_out1[pos1] += 1;”，即读一个 ram 内容，将其加一并写回。但可以在两个时钟周期，完成一个读写操作

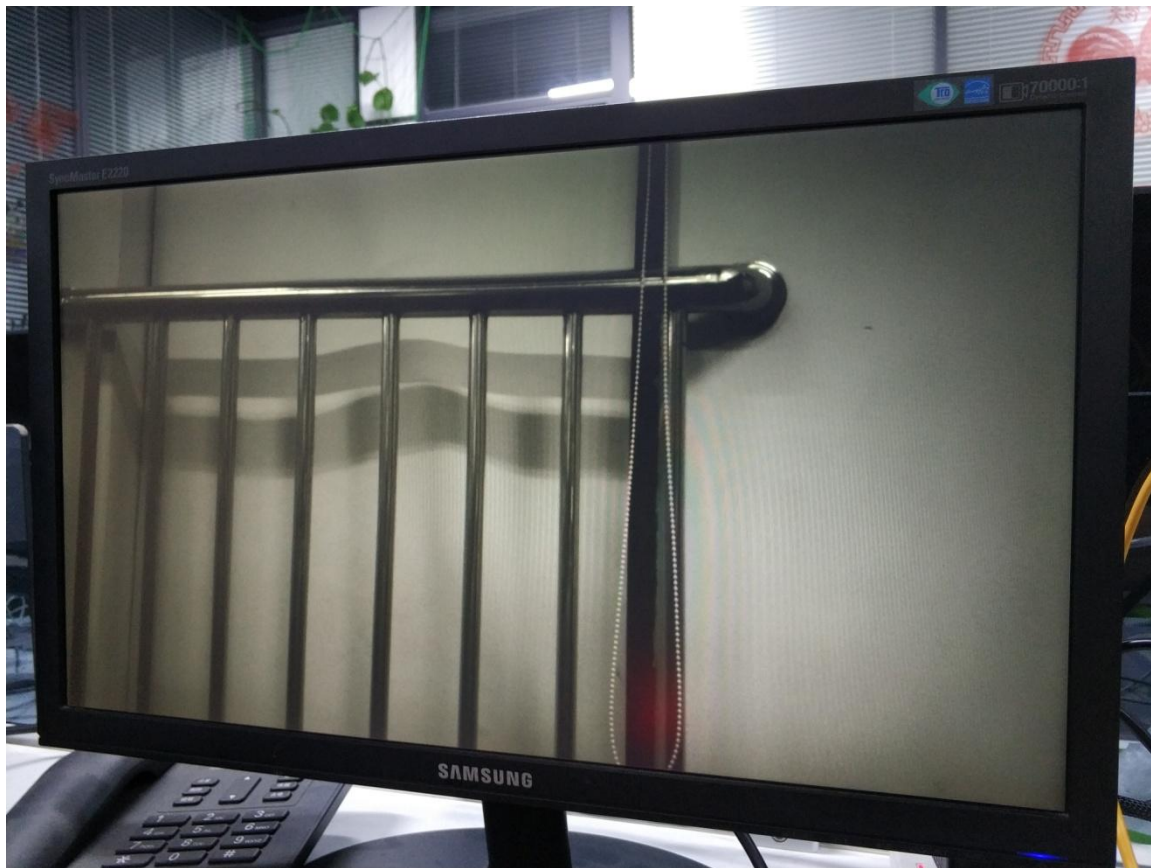
5.3 工程路径

名称	路径
Vivado 工程	vivado/contrast_adj
HLS 工程	hls/contrastadj
HLS 工程	hls/mem2stream
HLS 工程	hls/stream2mem
BOOT.bin 文件	bootimage

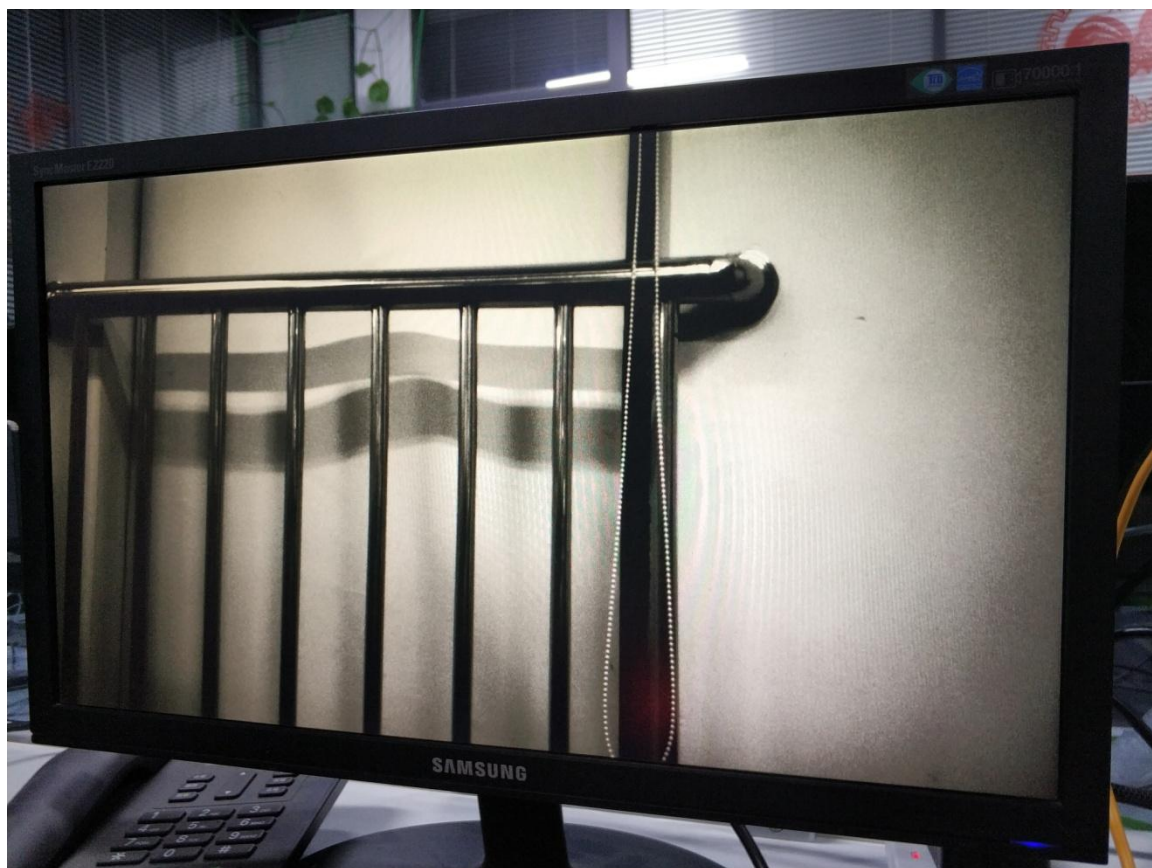
5.4 实验结果

对比度由 0 至 100 由 SDK 动态调整，0 为原图，100 为对比度最高。

对比度为 0 时



对比度为 100 时



第六章 边缘检测

6.1 实验介绍

Sobel 算子，也被称为 Sobel-Feldman 算子，或者 Sobel 滤波，是在图像处理和计算机视觉中得到广泛应用的一种图像边缘检测算法。它由斯坦福大学人工智能实验室（SAIL）的 Irwin Sobel 和 Gray Feldman 而得名。

Sobel 算子使用两个（3x3）矩阵来对原图进行卷积运算以计算出两个方向的灰度差分（偏导）的估计值（一个水平方向、一个竖直方向）。我们假定 A 是原始图像（彩色图像需先转换为灰度图像）， G_x 和 G_y 分别是在横向及纵向的灰度偏导的近似值（即两个方向上对原图的平面卷积结果）。数学表达如下：

$$\mathbf{G}_x = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} * \mathbf{A} \quad \text{and} \quad \mathbf{G}_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * \mathbf{A}$$

对应的计算过程如下：

$$G_x = [f(x+1,y-1) + 2*f(x,y-1) + f(x+1,y+1)] - [f(x-1,y-1) + 2*f(x,y-1) + f(x-1,y+1)]$$

$$G_y = [f(x-1,y-1) + 2f(x,y-1) + f(x+1,y-1)] - [f(x-1,y+1) + 2*f(x,y+1) + f(x+1,y+1)]$$

上式中， $f(x,y)$ 为图像 A 中 (x,y) 处的灰度值。由此便可以计算出每个点的 G_x 和 G_y 。

对于图像中的每个点，其梯度的估计值 G 便可以通过两个方向的梯度 G_x 和 G_y 借由下式得出：

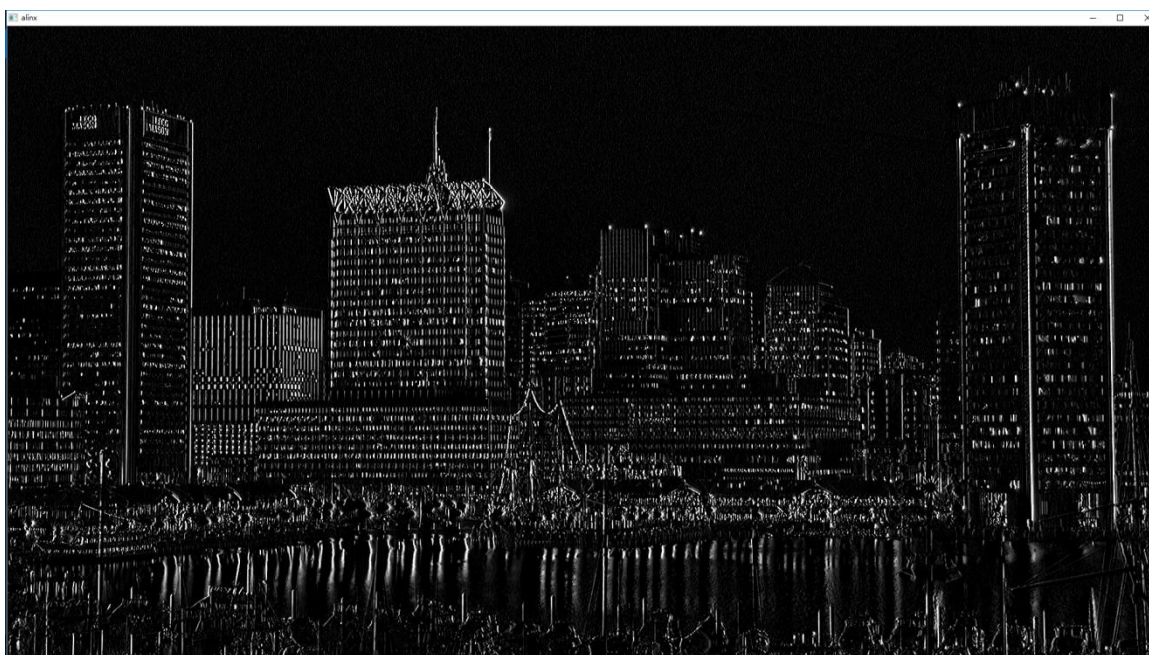
$$G = \sqrt{G_x^2 + G_y^2}$$

6.2 TestBench 结果

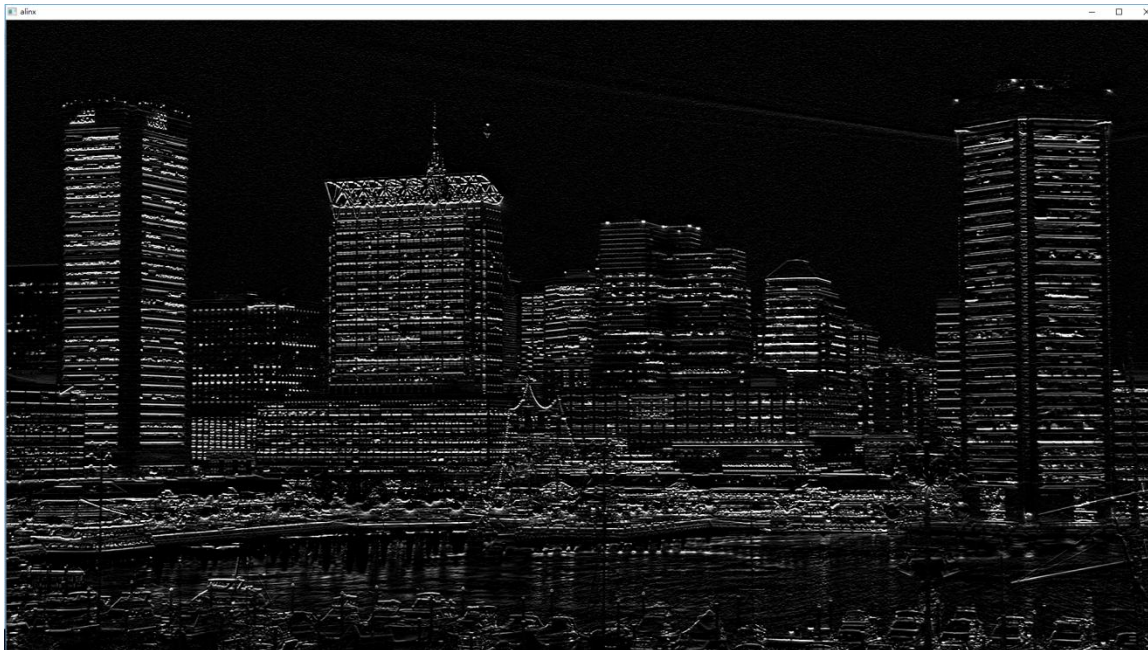
原始图片



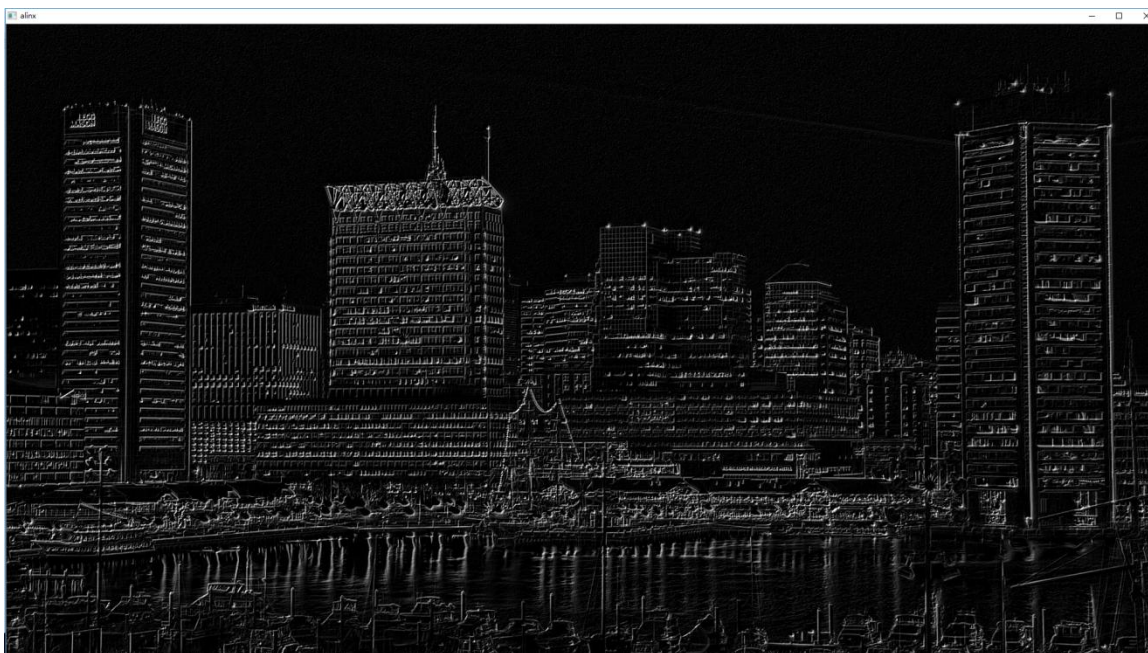
水平方向卷积运算结果



垂直方向卷积运算结果



最终运算结果



6.3 模块主要代码

```
#include "edge_detector.h"

void ExtractPixel(XF_TNAME(XF_8UC3,XF_NPPC1)&src,ap_uint<8>dst[3])
{
    unsigned int i,j=0;
    for(i=0;i<24;i+=8)
```



```
{
    dst[j]=src.range(i+7,i);
    j++;
}

template<int ROWS,int COLS>
void thresholding(xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS,
IMG_MAX_COLS,XF_NPPC1> &src,
                xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS,XF_NPPC1> &dst,
                ap_uint<8>&threshold)
{
    XF_TNAME(XF_8UC1,XF_NPPC1) pixel_src, pixel_dst;
    unsigned int i,j=0;
    ap_uint<8>value;
    if(threshold==0)
    {
        for(i=0;i<ROWS;i++)
        {
            for(j=0;j<COLS;j++)
            {
#pragma HLS PIPELINE
                pixel_src=src.read(i*COLS+j);
                dst.write(i*COLS+j,pixel_src);
            }
        }
    }
    else
    {
        for(i=0;i<ROWS;i++)
        {
            for(j=0;j<COLS;j++)
            {
#pragma HLS PIPELINE
                pixel_src=src.read(i*COLS+j);
                if(pixel_src>threshold)
                {
                    pixel_dst=0;
                }
                else
                {
                    pixel_dst=255;
                }
                dst.write(i*COLS+j,pixel_dst);
            }
        }
    }
}
```

```

template <int ROWS, int COLS>
void xfrgb2gray(xf::cv::Mat<XF_8UC3, ROWS, COLS, XF_NPPC1>& src,
               xf::cv::Mat<XF_8UC1, ROWS, COLS, XF_NPPC1>& dst)
{
    unsigned int i,j=0;
    XF_TNAME(XF_8UC3,XF_NPPC1)pixel_src;
    XF_TNAME(XF_8UC1,XF_NPPC1)pixel_dst;
    ap_uint<8>value[3];
    ap_uint<8>gray;
    for(i=0;i<ROWS;i++)
    {
        for(j=0;j<COLS;j++)
        {
            pixel_src=src.read(i*COLS+j);
            ExtractPixel(pixel_src,value);
            gray=CalculateGRAY(value[0],value[1],value[2]);
            pixel_dst.range(7,0)=gray;
            dst.write(i*COLS+j,pixel_dst);
        }
    }
}

template<int ROWS,int COLS>
void AddWeightedKernel(xf::cv::Mat<XF_8UC1,ROWS,COLS,XF_NPPC1>&src1,
                      float alpha,
                      xf::cv::Mat<XF_8UC1,ROWS,COLS,XF_NPPC1>&src2,
                      float beta,
                      float gamma,
                      xf::cv::Mat<XF_8UC1,ROWS,COLS,XF_NPPC1>&dst
                      )
{
    ap_fixed<16,8,AP_RND>value_src1=alpha;
    ap_fixed<16,8,AP_RND>value_src2=beta;
    ap_fixed<16,8,AP_RND>value_src3=gamma;
    XF_TNAME(XF_8UC1,XF_NPPC1)pixel1;
    XF_TNAME(XF_8UC1,XF_NPPC1)pixel2;
    XF_TNAME(XF_8UC1,XF_NPPC1)pixel3;
    ap_int<24>firstcmp;
    ap_int<24>secondcmp;
    ap_int<16>thirdcmp;
    ap_uint<8>value;
    ap_uint<8>value_cmp1;
    ap_uint<8>value_cmp2;
    unsigned int i,j=0;
    for(i=0;i<ROWS;i++)

```

```

    {
        for(j=0;j<COLS;j++)
        {
#pragma HLS pipeline
            pixel1=src1.read(i*COLS+j);
            pixel2=src2.read(i*COLS+j);
            value_cmp1=pixel1.range(7,0);
            value_cmp2=pixel2.range(7,0);
            firstcmp=(ap_int<24>)value_cmp1*value_src1;
            secondcmp=(ap_int<24>)value_cmp2*value_src2;
            thirdcmp=(ap_int<16>)firstcmp+secondcmp+value_src3;
            if(thirdcmp>255)
            {
                thirdcmp=255;
            }
            else if(thirdcmp<0)
            {
                thirdcmp=0;
            }
            value=thirdcmp;
            pixel3.range(7,0)=value;
            dst.write(i*COLS+j,pixel3);
        }
    }
}

template<int ROWS,int COLS>
void
xfgray2rgb(xf::cv::Mat<XF_8UC1,ROWS,COLS,XF_NPPC1>&src,xf::cv::Mat<XF_8UC3
,ROWS,COLS,XF_NPPC1> &dst)
{
    unsigned int i,j=0;
    XF_TNAME(XF_8UC1,XF_NPPC1)temp_src;
    XF_TNAME(XF_8UC3,XF_NPPC1)temp_dst;
    ap_uint<8>value;
    for(i=0;i<ROWS;i++)
    {
        for(j=0;j<COLS;j++)
        {
#pragma HLS PIPELINE
            temp_src=src.read(i*COLS+j);
            value=temp_src.range(7,0);
            temp_dst.range(7,0)=value;
            temp_dst.range(15,8)=value;
            temp_dst.range(23,16)=value;
            dst.write(i*COLS+j,temp_dst);
        }
    }
}

```

```

}

void edge_detector(ustream_t &src, ustream_t &dst, ap_uint<8> threshold)
{
#pragma HLS DATAFLOW
#pragma HLS INTERFACE axis port=src
#pragma HLS INTERFACE axis port=dst
#pragma HLS INTERFACE mode=s_axilite port=threshold register
#pragma HLS INTERFACE mode=ap_ctrl_none port=return

    xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS, IMG_MAX_COLS,XF_NPPC1> srcImg;
#pragma HLS STREAM depth=1920 type=fifo variable=srcImg
    xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS, IMG_MAX_COLS,XF_NPPC1> rgbSobel;
#pragma HLS STREAM depth=1920 type=fifo variable=rgbSobel
    xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS,XF_NPPC1> grayImg;
#pragma HLS STREAM depth=1920 type=fifo variable=grayImg
    xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS,XF_NPPC1> sobelImg_x;
#pragma HLS STREAM depth=1920 variable=sobelImg_x
    xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS,XF_NPPC1> sobelImg_y;
#pragma HLS STREAM depth=1920 type=fifo variable=sobelImg_y
    xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS,XF_NPPC1> sobelImg;
#pragma HLS STREAM depth=1920 type=fifo variable=sobelImg
    xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS,XF_NPPC1>
thresholdImg;
#pragma HLS STREAM depth=1920 variable=thresholdImg

    xf::cv::AXIvideo2xfMat(src, srcImg);
    xfrgb2gray<IMG_MAX_ROWS,IMG_MAX_COLS>(srcImg, grayImg);
    xf::cv::xFSobelFilter3x3<XF_8UC1, XF_8UC1,IMG_MAX_ROWS,
IMG_MAX_COLS, XF_CHANNELS(XF_8UC1,XF_NPPC1), XF_DEPTH(XF_8UC1,XF_NPPC1),
XF_DEPTH(XF_8UC1,XF_NPPC1),
XF_NPPC1,_XFCVDEPTH_DEFAULT,_XFCVDEPTH_DEFAULT,_XF
CVDEPTH_DEFAULT,XF_WORDWIDTH(XF_8UC1,XF_NPPC1),
XF_WORDWIDTH(XF_8UC1,XF_NPPC1), (IMG_MAX_COLS >>
XF_BITSHIFT(XF_NPPC1)),false>(
    grayImg,sobelImg_x,sobelImg_y,grayImg.rows,grayImg.cols>>XF_B
ITSHIFT(XF_NPPC1));
    AddWeightedKernel<IMG_MAX_ROWS,IMG_MAX_COLS>(sobelImg_x,0.5f,sobelI
mg_y,0.5f,0.0f,sobelImg);
    thresholding<IMG_MAX_ROWS,IMG_MAX_COLS>(sobelImg,
thresholdImg,threshold);
    xfgray2rgb<IMG_MAX_ROWS,IMG_MAX_COLS>(thresholdImg, rgbSobel);
    xf::cv::xfMat2AXIvideo(rgbSobel, dst);
}

```

这里 threshold 为将结果二值化的配置，为 0 时不进行二值化，其它值时为二值化的阈值。我们首选将 rgb 图像转换为灰度图 grayImg，将 grayImg 分成两份 split0、split1 分别进行水平和垂直方向的卷积运算，得到 sobelImg_x 和 sobelImg_y，将它们叠加为 sobelImg。然后将其二值

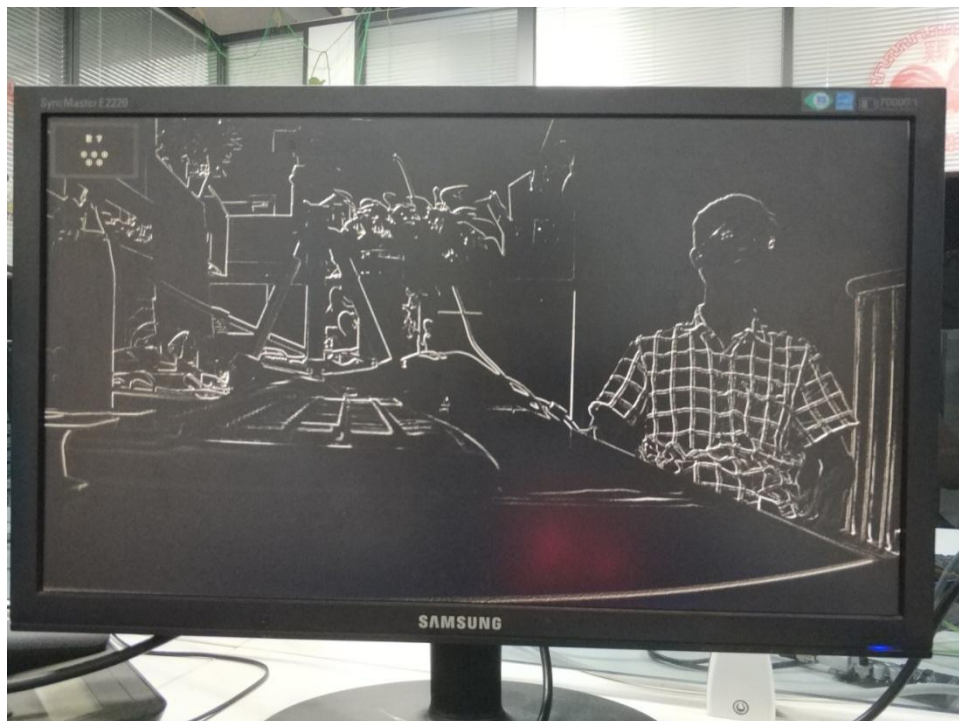
化, 转为 RGB, 输出。

6.4 工程路径

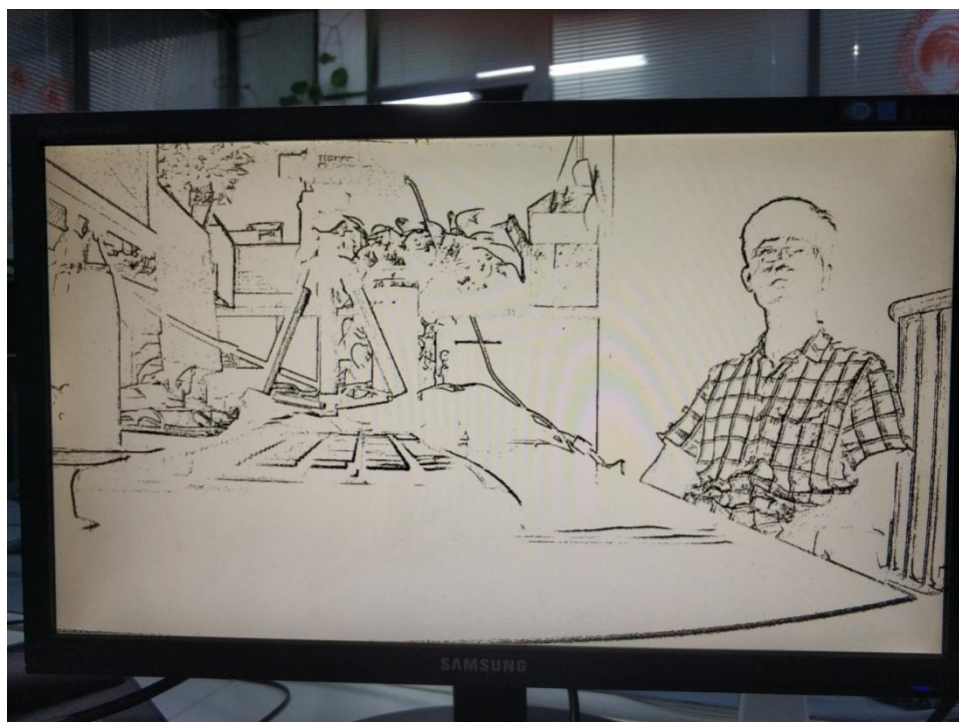
名称	路径
Vivado 工程	vivado/ edge_detector
HLS 工程	hls/ edge_detector
BOOT.bin 文件	bootimage

6.5 实验结果

Threshold 设置为 0 时



Threshold 设置为 20 时



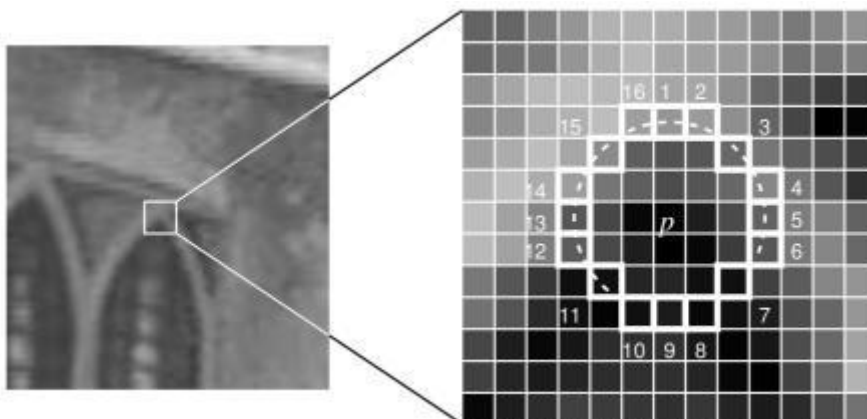
第七章 角点检测

角点检测(Corner Detection)是计算机视觉系统中用来获得图像特征的一种方法,广泛应用于运动检测、图像匹配、视频跟踪、三维建模和目标识别等领域中。也称为特征点检测。

FAST (Features from accelerated segment test) 是一种角点检测方法,最初是由 Edward Rosten 和 Tom Drummond 提出,该算法最突出的优点是它的计算效率。正如它的缩写名字,它很快而且事实上它比其他著名的特征点提取方法(如 SIFT, SUSAN, Harris)都要快。如果应用于机器学习方法的话,该算法能够取得更佳的效果。正因为它的快速特点,FAST 角点检测方法非常适用于实时视频处理的领域。

FAST 算法步骤

- 1) 从图片中选取一个像素 P , 下面我们将判断它是否是一个特征点。我们首先把它的亮度值设为 I_p 。
- 2) 设定一个合适的阈值 t 。
- 3) 考虑以该像素点为中心的一个半径等于 3 像素的离散化的 Bresenham 圆, 这个圆的边界上有 16 个像素(如图 1 所示)。



- 4) 现在,如果在这个大小为 16 个像素的圆上有 n 个连续的像素点,它们的像素值要么都比 I_p+t 大,要么都比 I_p-t 小,那么它就是一个角点。(如图 1 中的白色虚线所示)。 n 的值可以设置为 12 或者 9, 实验证明选择 9 可能会有更好的效果。

上面的算法中,对于图像中的每一个点,我们都要去遍历其邻域圆上的 16 个点的像素,效率较低。我们下面提出了一种高效的测试 (high-speed test) 来快速排除一大部分非角点的像素。该方法仅仅检查在位置 1, 9, 5 和 13 四个位置的像素,首先检测位置 1 和位置 9, 如果它们都比阈值暗或比阈值亮,再检测位置 5 和位置 13。如果 P 是一个角点,那么上述四个像素点中至少有 3 个应该必须都大于 I_p+t 或者小于 I_p-t , 因为若是一个角点,超过四分之三圆的部分应该满足判断条件。如果不满足,那么 P 不可能是一个角点。对于所有点做上面这一部分初步的检测后,符合条件的将成为候选的角点,我们再对候选的角点,做完整的测试,即检测圆上的所有点。

上面的算法效率实际上是很高的,但是有一些缺点:

- 1) 当我们设置 $n < 12$ 时就不能使用快速算法来过滤非角点的点;

- 2) 检测出来的角点不是最优的，这是因为它的效率取决于问题的排序与角点的分布；
- 3) 对于角点分析的结果被丢弃了；
- 4) 多个特征点容易挤在一起。

可以通过机器学习和非极大值抑制的方法来增强鲁棒性。由于 opencv 中相关的函数没有使用机器学习，因此我们这里只介绍非极大值抑制的方法。由于分割测试并没有计算角点响应函数，因此常规的非极大值抑制方法并不适用于 FAST 算法。下面是 FAST 的非极大值抑制方法：

- 1) 计算得分函数，它的值 v 是特征点与其圆周上 16 个像素点的绝对差值中所有连续 10 个像素中的最小值的最大值，而且该值还要大于阈值 t ；
- 2) 在 3×3 的特征点邻域内（而不是图像邻域），比较 v ；
- 3) 剔除掉非极大值的特征点。

在 opencv 中，实现 FAST 算法的核心函数有两个，它们的原型为：

```
void FAST(InputArray image, vector<KeyPoint>& keypoints, int threshold, bool
nonmaxSuppression=true )
```

```
void FASTX(InputArray image, vector<KeyPoint>& keypoints, int threshold, bool nonmaxSuppression,
int type)
```

image 为输入图像，要求是灰度图像，keypoints 为检测到的特征点向量，threshold 为阈值 t ，nonmaxSuppression 为是否进行非极大值抑制，true 表示进行非极大值抑制，type 为选取圆周像素点的个数，是 8 (FastFeatureDetector::TYPE_5_8)、12 (FastFeatureDetector::TYPE_7_12) 还是 16 (FastFeatureDetector::TYPE_9_16)。该参数是 FAST 函数和 FASTX 函数的区别，事实上，FAST 函数是调用 FASTX 函数，而传入的 type 值为 FastFeatureDetector::TYPE_9_16。

本节实验，我们创建一个 detectCorner 的 HLS 模块，从 DDR 读取视频图像，检测角点后，将结果写回 DDR。

在 SDK 中，我们将写回 DDR 的角点检测结果，叠加在显示的图像上。

7.1 模块主要代码

```
#include "detectCorner.h"

void ExtractPixel(XF_TNAME(XF_8UC3,XF_NPPC1)&src,ap_uint<8> value[3])
{
#pragma HLS INLINE off
    unsigned i,j=0;
    for(i=0;i<24;i+=8)
    {
#pragma HLS UNROLL
        value[j]=src.range(i+7,i);
        j++;
    }
}
```

```

void PackPixel(XF_TNAME(XF_8UC3,XF_NPPC1)&dst,ap_uint<8> value[3])
{
#pragma HLS INLINE off
    unsigned int i,j=0;
    for(i=0;i<24;i+=8)
    {
#pragma HLS UNROLL
        dst.range(i+7,i)=value[j];
        j++;
    }
}

template<int ROWS,int COLS>
void
xfrgb2gray(xf::cv::Mat<XF_8UC3,ROWS,COLS,XF_NPPC1>&src,xf::cv::Mat<XF_8UC1
,ROWS,COLS,XF_NPPC1>&dst1,xf::cv::Mat<XF_8UC3,ROWS,COLS,XF_NPPC1>&dst2)
{
    XF_TNAME(XF_8UC3,XF_NPPC1)pixel_src;
    XF_TNAME(XF_8UC1,XF_NPPC1)pixel_dst1;
    XF_TNAME(XF_8UC3,XF_NPPC1)pixel_dst2;
    ap_uint<8>rgb[3];
    ap_uint<8>gray;
    unsigned int i,j=0;
    for(i=0;i<ROWS;i++)
    {
        for(j=0;j<COLS;j++)
        {
            pixel_src=src.read(i*COLS+j);
            ExtractPixel(pixel_src,rgb);
            gray=CalculateGRAY(rgb[0],rgb[1],rgb[2]);
            pixel_dst1.range(7,0)=gray;
            pixel_dst2=pixel_src;
            dst1.write(i*COLS+j,pixel_dst1);
            dst2.write(i*COLS+j,pixel_dst2);
        }
    }
}

template<int ROWS,int COLS>
void
xfgray2rgb(xf::cv::Mat<XF_8UC1,ROWS,COLS,XF_NPPC1>&src1,xf::cv::Mat<XF_8UC
3,ROWS,COLS,XF_NPPC1>&src2,xf::cv::Mat<XF_8UC3,ROWS,COLS,XF_NPPC1>&dst)
{
    unsigned int i,j=0;
    XF_TNAME(XF_8UC1,XF_NPPC1)pixel_src1;
    XF_TNAME(XF_8UC3,XF_NPPC1)pixel_src2;
    XF_TNAME(XF_8UC3,XF_NPPC1)pixel_dst;

```

```

        for(i=0;i<ROWS;i++)
        {
            for(j=0;j<COLS;j++)
            {
                pixel_src1=src1.read(i*COLS+j);
                pixel_src2=src2.read(i*COLS+j);
                if(pixel_src1==255)
                {
                    pixel_dst.range(7,0)=0x00;
                    pixel_dst.range(15,8)=pixel_src1;
                    pixel_dst.range(23,16)=0x00;
                }
                else
                {
                    pixel_dst=pixel_src2;
                }
                dst.write(i*COLS+j,pixel_dst);
            }
        }
    }

    void detectCorner(hls::stream<ap_axiu<24,1,1,1>>&video_in,
hls::stream<ap_axiu<24,1,1,1>>&video_out,int threshold)
    {
        #pragma HLS INTERFACE mode=s_axilite port=threshold register
        #pragma HLS INTERFACE axis port=video_out register_mode=both register
        #pragma HLS INTERFACE axis port=video_in register_mode=both register
        #pragma HLS INTERFACE ap_ctrl_none port=return
        #pragma HLS DATAFLOW

        xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1> img_in;
        #pragma HLS STREAM depth=1920 type=fifo variable=img_in
        xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1> img_out;
        #pragma HLS STREAM depth=1920 type=fifo variable=img_out
        xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1>
img_rgb_src;
        #pragma HLS STREAM depth=1920 type=fifo variable=img_rgb_src
        xf::cv::Mat<XF_8UC3,IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1>
img_rgb_dst;
        #pragma HLS STREAM depth=1920 type=fifo variable=img_rgb_dst
        xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1>
img_gray_src;
        #pragma HLS STREAM depth=1920 type=fifo variable=img_gray_src
        xf::cv::Mat<XF_8UC1,IMG_MAX_ROWS, IMG_MAX_COLS, XF_NPPC1>
img_gray_dst;
        #pragma HLS STREAM depth=1920 type=fifo variable=img_gray_dst

```

```

unsigned char kernel[NEW_K_ROWS][NEW_K_COLS];
#pragma HLS array_partition variable=kernel dim=0
// clang-format on
for (unsigned char i = 0; i < NEW_K_ROWS; i++) {
    for (unsigned char j = 0; j < NEW_K_COLS; j++) {
        kernel[i][j] = 1; // _kernel[i*NEW_K_COLS+j];
    }
}

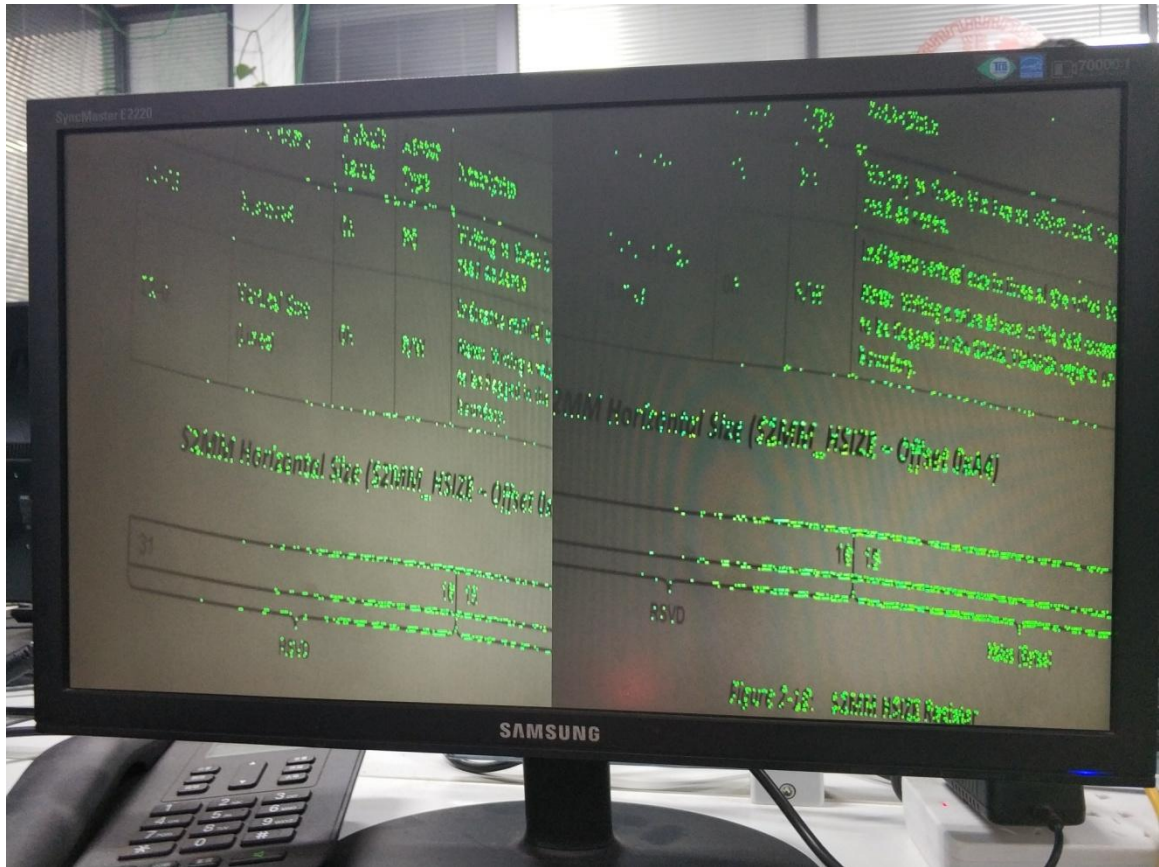
xf::cv::AXIvideo2xfMat(video_in,img_in);
xfrgb2gray<IMG_MAX_ROWS,IMG_MAX_COLS>(img_in,img_gray_src,img_rgb_s
rc);
xf::cv::fast<0,XF_8UC1,IMG_MAX_ROWS,IMG_MAX_COLS,XF_NPPC1>(img_gray
_src,img_rgb_src,img_gray_dst,img_rgb_dst,threshold);
//    xf::cv::xfdilate<IMG_MAX_ROWS,IMG_MAX_COLS, XF_CHANNELS(XF_8UC1,
XF_NPPC1), XF_8UC1, XF_NPPC1, 0, (IMG_MAX_COLS >> XF_BITSHIFT(XF_NPPC1)) +
(NEW_K_ROWS >> 1),
//                                NEW_K_ROWS,
NEW_K_COLS>(mask,dmask,matGray.rows,matGray.cols>>XF_BITSHIFT(XF_NPPC1),ke
rnel);
//    overlyOnMat<IMG_MAX_ROWS,IMG_MAX_COLS>(img_rgb_dst,img_gray_dst
,img_out,overly_alpha,overly_x,overly_y,overly_h,overly_w);
xfgray2rgb<IMG_MAX_ROWS,IMG_MAX_COLS>(img_gray_dst,img_rgb_dst,img_
out);
xf::cv::xfMat2AXIvideo(img_out,video_out);
}

```

7.2 工程路径

名称	路径
Vivado 工程	vivado/dual_corner
HLS 工程	hls/dual_corner
HLS 工程	hls/mem2stream
HLS 工程	hls/stream2mem
BOOT.bin 文件	bootimage

7.3 实验结果



这里角点多少与聚焦位置及阈值有关。需要根据场景调整这两个参数。